

CHAPTER 0

JAVA

PROGRAMMING

FUNDAMENTALS

OOP PREPARATORY KNOWLEDGE & SKILLS

1

June Arreb C. Danila, MIT



INTENDED LEARNING OUTCOMES

- ❑ Identify the basic program structure, elements, and control structures of a Java program *[Introductory]*
- ❑ Determine how to use predefined classes, string object, and wrapper classes *[Enabling]*
- ❑ Create program solutions using the java programming basics and fundamentals *[Demonstrative]*

CHAPTER 1A

JAVA

PROGRAMMING

BASICS AND FLOW

CONTROLS



REVIEW OF SOME PROGRAMMING CONCEPTS

4



WHAT IS

PROGRAMMING?

- ❑ **Programming** is the act or job of creating computer programs (*Merriam-Webster Dictionary*)
- ❑ **Programming** is the act or process of writing a program so that data may be processed by a computer (*Collins Dictionary*)
- ❑ **Programming** is the implementation of logic to facilitate specified computing operations and functionality (*techopedia.com*)

WHAT IS PROGRAM?

- ❑ **Program** is a sequence of coded instructions that can be inserted into a mechanism such as a computer (*Merriam-Webster Dictionary*)
- ❑ **Program** is a specific set of ordered operations for a computer to perform. (*WhatIs.com*)
- ❑ **Program** is a set of instructions that a computer follows in order to perform a particular task (*Collins Dictionary*)

WHAT IS

SOURCE CODE?

- ❑ **Source code** is the original form of a computer program before it is converted into a machine-readable code(*Collins Dictionary*)
- ❑ **Source code** referred to as simply the "***source***" of a ***program***, contains variable declarations, instructions, functions, loops, and other statements that tell the program how to function (*techterms.com*)
- ❑ **Source code** is the set of instructions and statements written by a programmer using a computer programming language (*techopedia.com*)

WHAT IS

PROGRAMMING LANGUAGE?

- ❑ **Programming Language** is the language in which a programmer writes the instructions that the computer will ultimately execute (*britannica.com*)
- ❑ **Programming Language** is a vocabulary and set of grammatical rules for instructing a computer or computing device to perform specific tasks (*webopedia.com*)
- ❑ **Programming Language** is a set of commands, instructions, and other syntax use to create a software program (*techterms.com*)

WHAT ARE THE

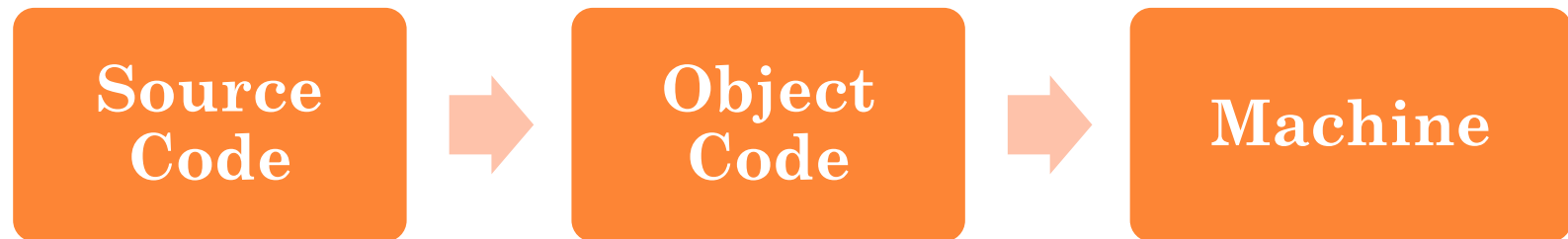
TYPES OF PROG. LANGUAGES?

- ▣ **Compiled Language**
- ▣ **Interpreted Language**
- ▣ **Bytecode Language**

Source: thesocietea.org

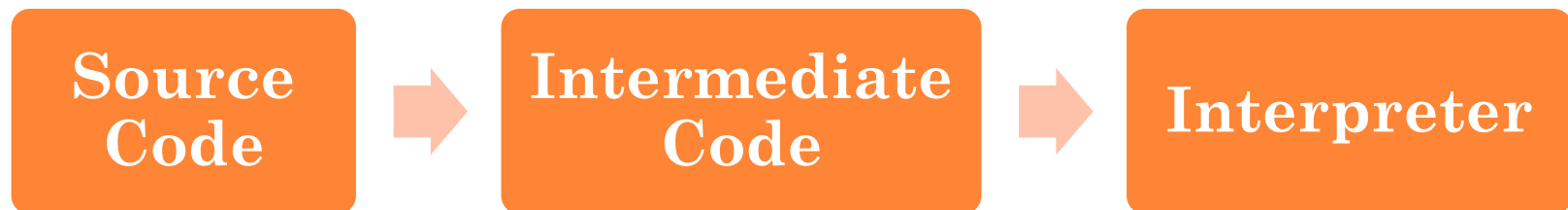
WHAT IS COMPILED LANGUAGE?

- ▣ **Compiled Language** is one where the program, once compiled, is expressed in the instructions of the target machine; this machine code is undecipherable by humans



WHAT IS INTERPETED LANGUAGE?

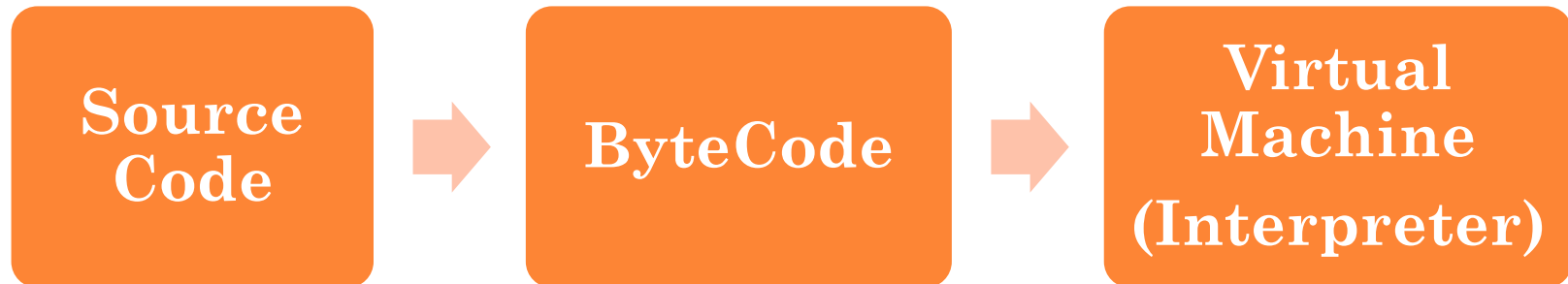
- ▣ **Interpreted Language** is one where the instructions are not directly executed by the target machine, but instead read and executed by some other program (which normally *is* written in the language of the native machine)



WHAT IS

BYTECODE LANGUAGE?

- ▣ **Bytecode Language** is a type of programming language that fall under the categories of both compiled and interpreted languages because they employ both compilation and interpretation to execute code



Source: thesocietea.org

- ▣ **Bytecode** is a form of instruction set that is designed to be efficiently executed by an interpreter and is composed of compact numeric codes, constants, and memory references
- ▣ *Example of Bytecode Language: JAVA*

JAVA TECHNOLOGY

14



THE JAVA TECHNOLOGY

- ❑ Java is the global standard for developing and delivering embedded and mobile applications, games, web-based content, and enterprise software
- ❑ Java enables you to efficiently develop, deploy, and use exciting applications and services
- ❑ From laptops to data centers, game consoles to scientific supercomputers, cell phones to the Internet, ***JAVA IS EVERYWHERE!***

Source: Oracle Academy



- Java is the single most widely used development language in the world today
- Over 9 million Java developers
- Over 7 Billion devices used Java
 - Desktops
 - Mobile Phones
 - Appliances
 - Cards
 - Clouds
 - AI & Analytics

Source: Oracle Academy

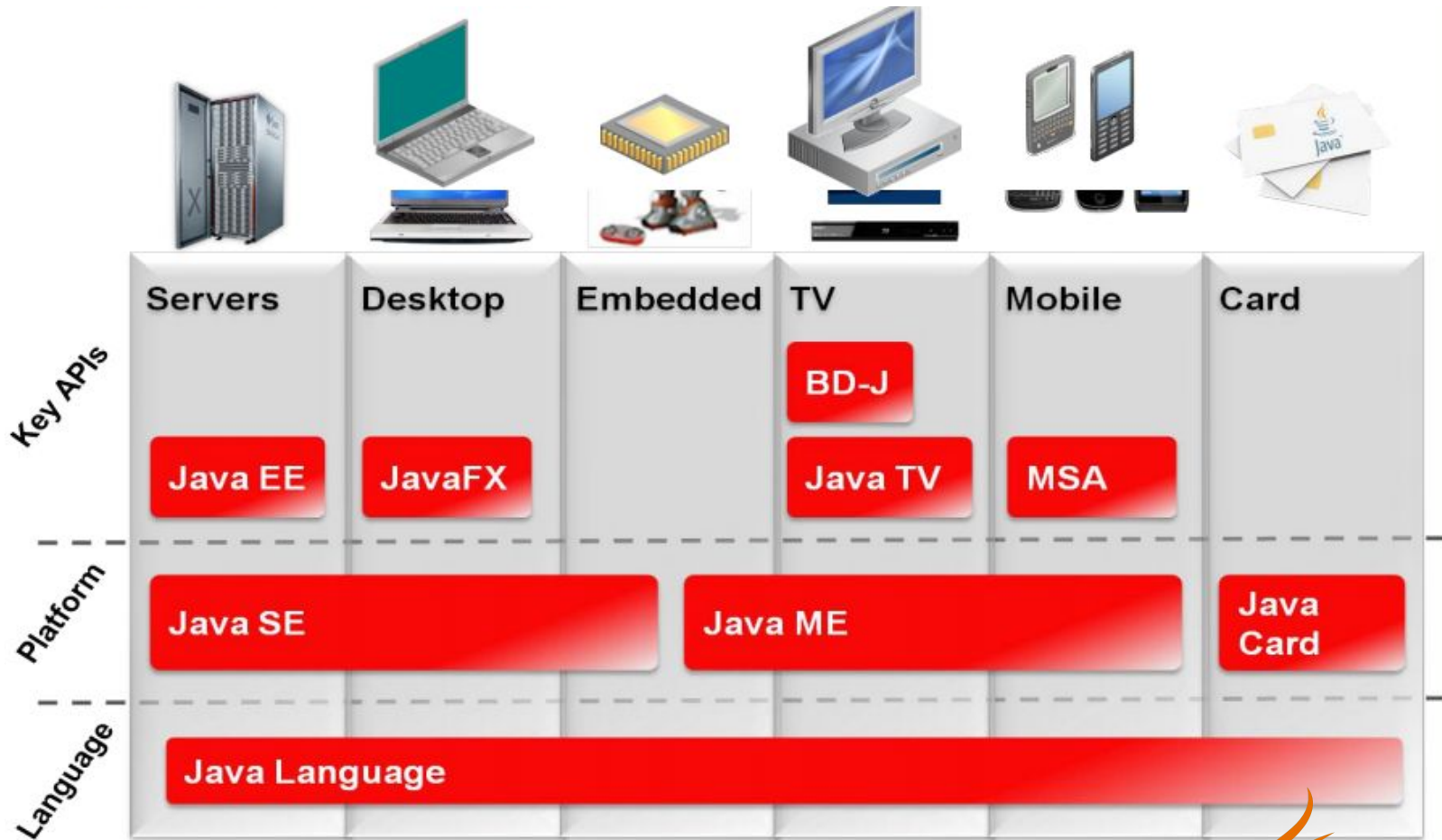


JAVA TECHNOLOGY PRODUCT

- Java Platform
 - Standard Edition (*Java SE*)
 - Enterprise Edition (*Java EE*)
 - Micro Edition (*Java ME*)
- Java Card



JAVA TARGET DEVICE TYPE

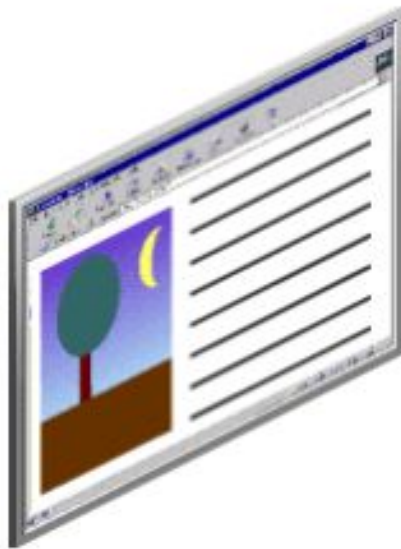


Source: Oracle Academy



JAVA SE

- Is used to develop applications that run on desktop computers

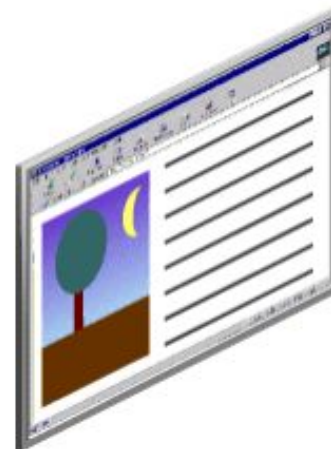


Source: Oracle Academy



JAVA EE

- ❑ Is used to create large enterprise, server-side, and client-side distributed applications



Source: Oracle Academy



JAVA ME

- ❑ Is used to create applications for devices with limited storage, display, and power capacities.
- ❑ Is used to develop applications for mobile phones, PDAs, TV set-top boxes, smart cards, Raspberry Pi, and many more.



Source: Oracle Academy



JAVA CARD

- ❑ 5 billion Java Cards are in use
- ❑ It's used to create applications that can run securely on smart cards and similar small-memory devices
- ❑ Java Card is typically used in the following areas (and many more):
 - Identity
 - Security
 - Transactions
 - Mobile phone SIMs



Source: Oracle Academy

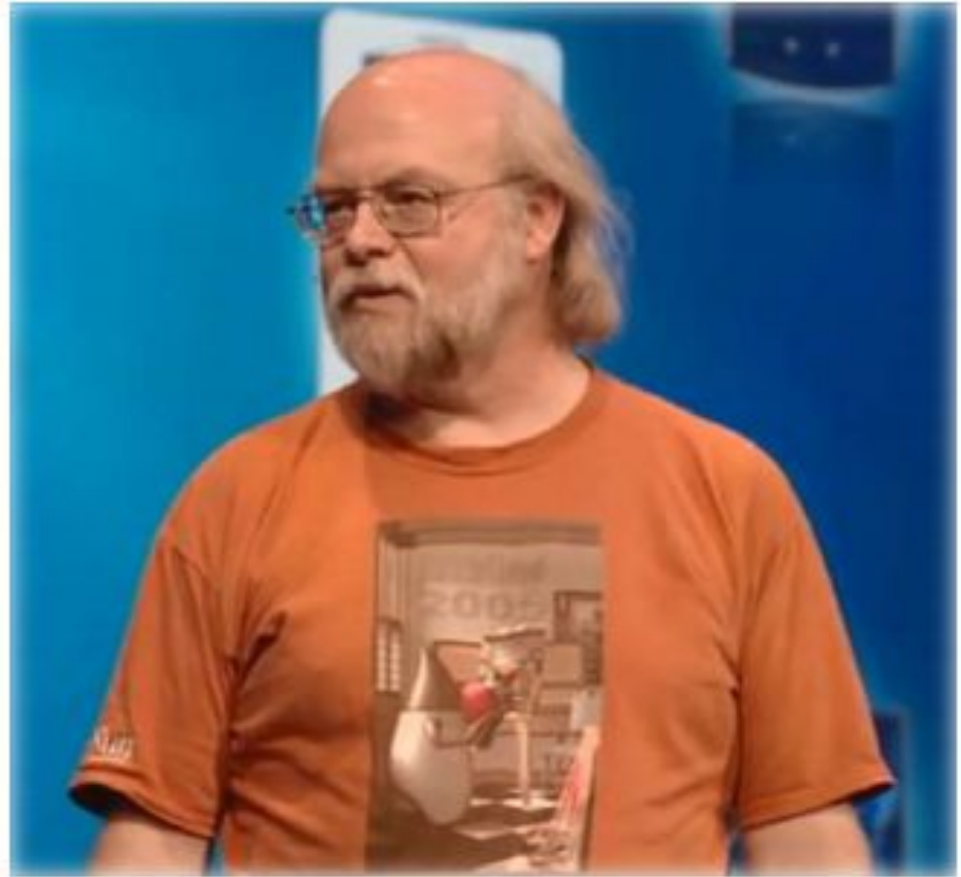
JAVA: BRIEF HISTORY

- ❑ In 1990, *Sun Microsystems* began a research project to extend the power of network computing to consumer devices, such as *video cassette recorders (VCRs)* and *televisions*
- ❑ The belief was that the next wave in computing was the union of digital consumer devices and computers
- ❑ There were also *frustrations* with the use of the *C/C++ language* at



Source: Oracle Academy

- ❑ The ***Green Team***, a team of highly skilled software developers at Sun under the leadership of ***James Gosling***, developed Java (originally called ***Oak***) as their solution.



James Gosling is considered the "Father of Java."

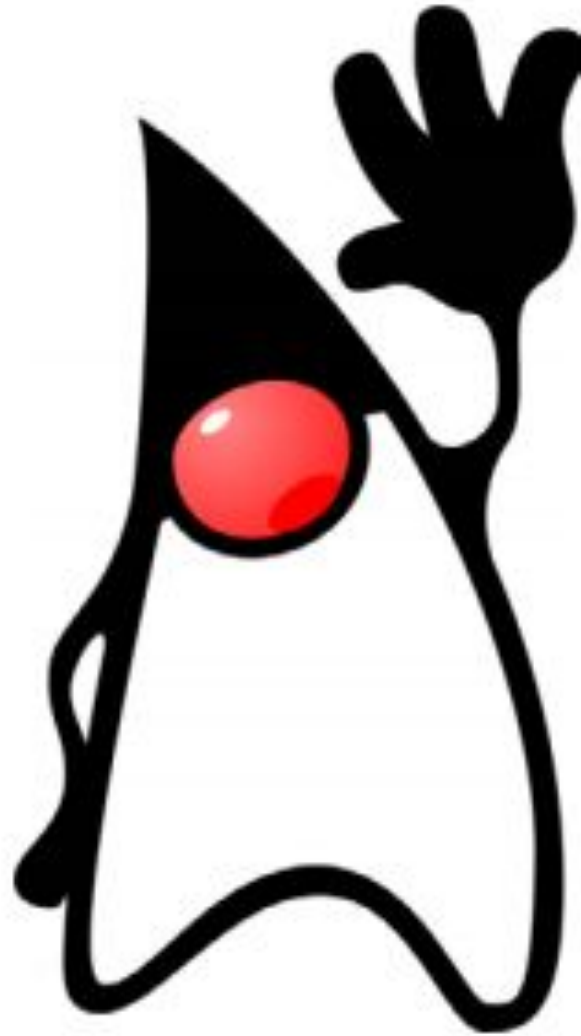
- ❑ Devices with different central processing units (CPUs) could be connected and share the same software enhancements through a single programming language
- ❑ This initial concept was ahead of its time, as several deals with consumer device companies were unsuccessful
- ❑ The Green Team was forced to find another market for their new programming language.

- ❑ Fortunately, the *World Wide Web* was becoming popular and the Green Team recognized that the Oak language was perfect for developing web multimedia components to enhance web pages
- ❑ Initially, the *Oak* language was used for small applications, called *applets*, and programmers using the Internet adopted what eventually became the *Java Programming language*

- ❑ The turning point for Java came in **1995**, when ***Netscape*** incorporated Java into its browser
- ❑ **Oracle** acquired Sun Microsystems in 2010

The Oracle logo is displayed in a large, bold, red, sans-serif font. The word "ORACLE" is written in all caps, with a registered trademark symbol (®) to the upper right of the final "E".

- ▣ ***Duke*** is Java's official mascot
- ▣ The original Duke was created by the Green Team's graphic artist, ***Joe Palrang***



Java Version	Released
JDK Alpha & Beta	1995
JDK 1.0	1996
JDK 1.1	1997
JDK 1.2	1998
JDK 1.4	2000
JDK 5	2004
JDK 6	2006
JDK 7	2011
JDK 8 (LTS)	March, 2014
JDK 9	September, 2017
JDK 10	March, 2018
JDK 11 (LTS)	September, 2018



Source: Oracle Academy



JAVA ENVIRONMENT (JVM, JRE, JDK, IDE)

31

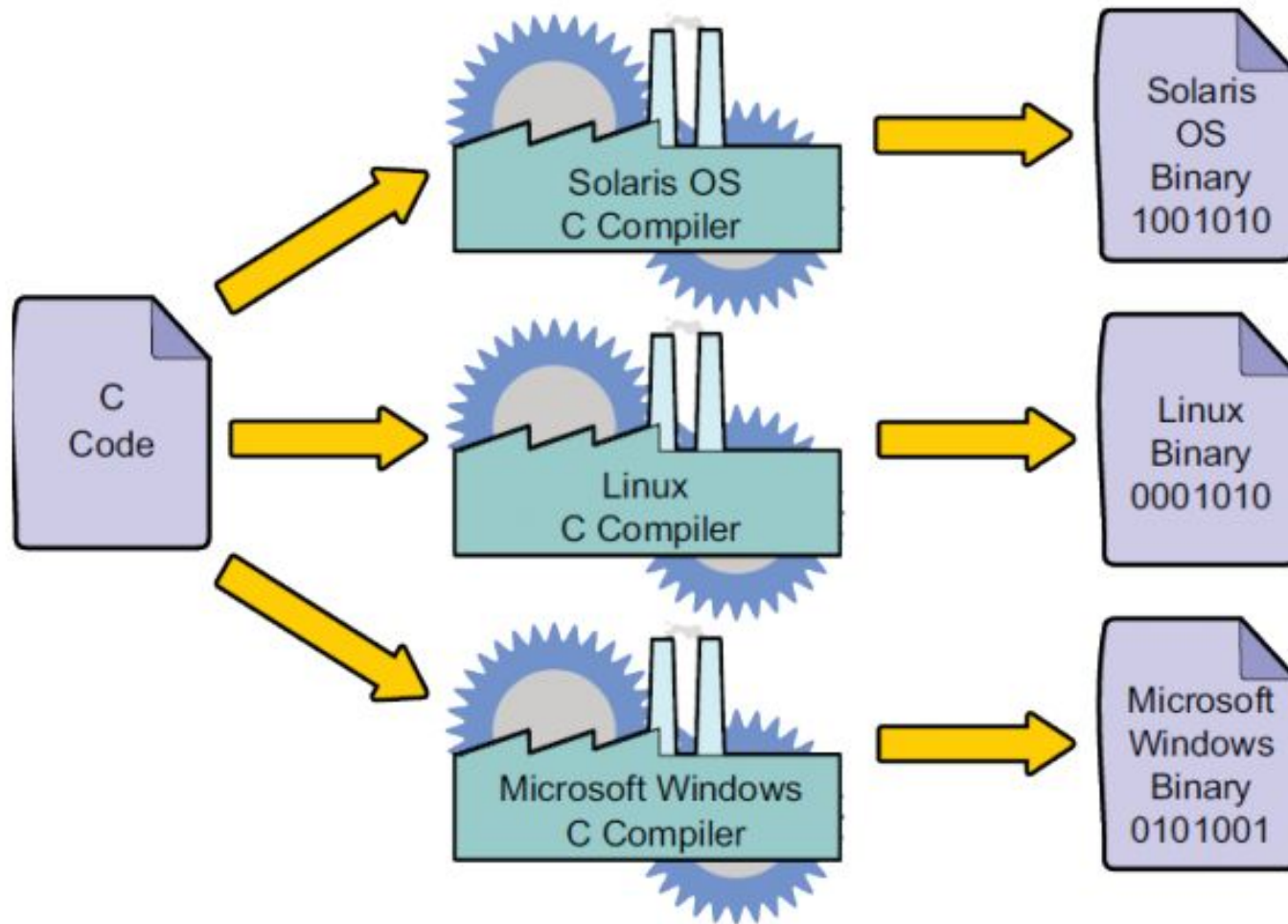


PURPOSE OF A COMPUTER PROGRAM

- A computer program is a set of instructions that run on a computer or other digital device
 - At the machine level, the program consists of binary instructions (1s and 0s), called *Machine code*
 - Most programs are written in *high-level code* (readable), must be translated to machine code



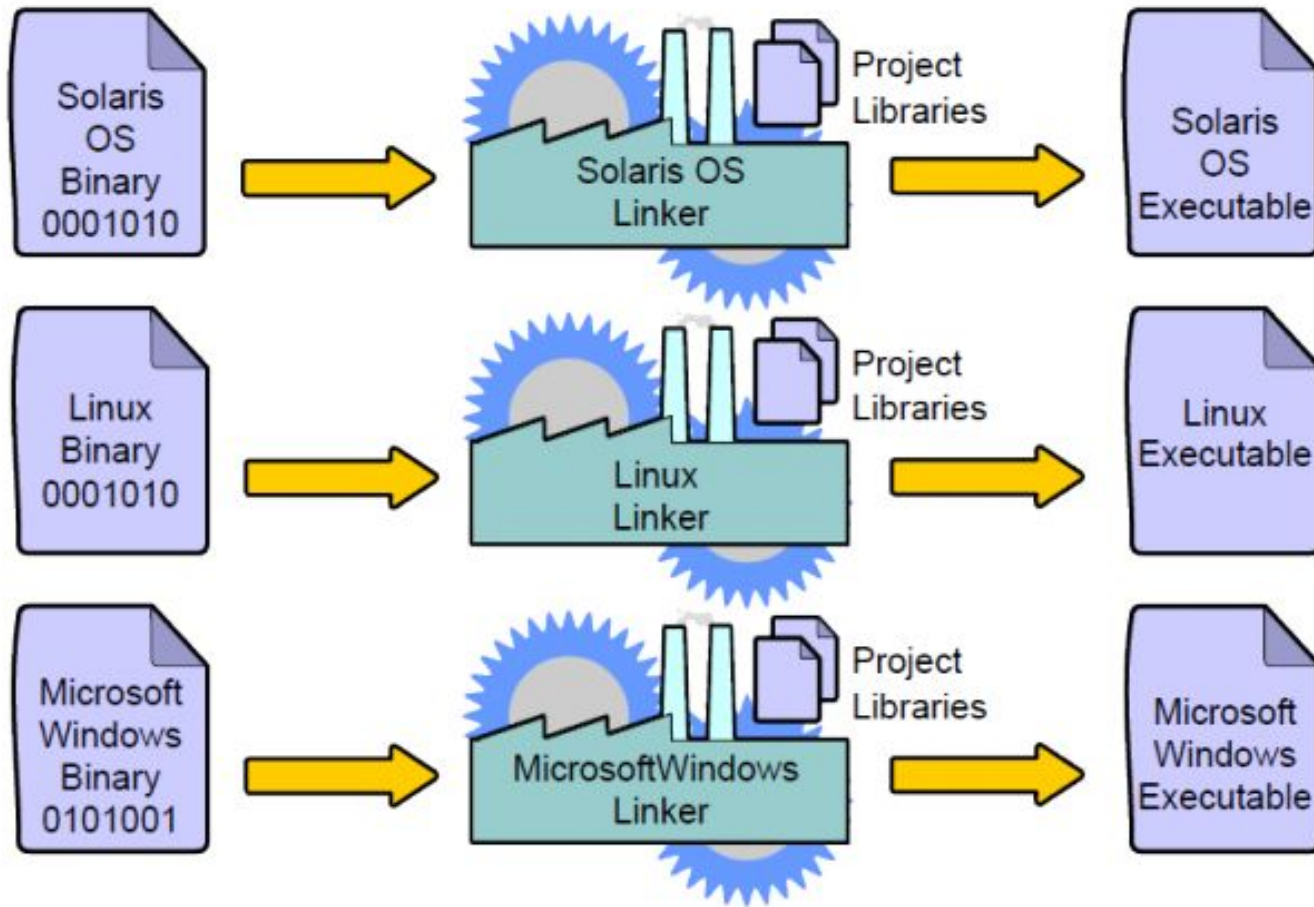
TRANSLATING HIGH-LEVEL CODE TO MACHINE CODE



Source: Oracle Academy



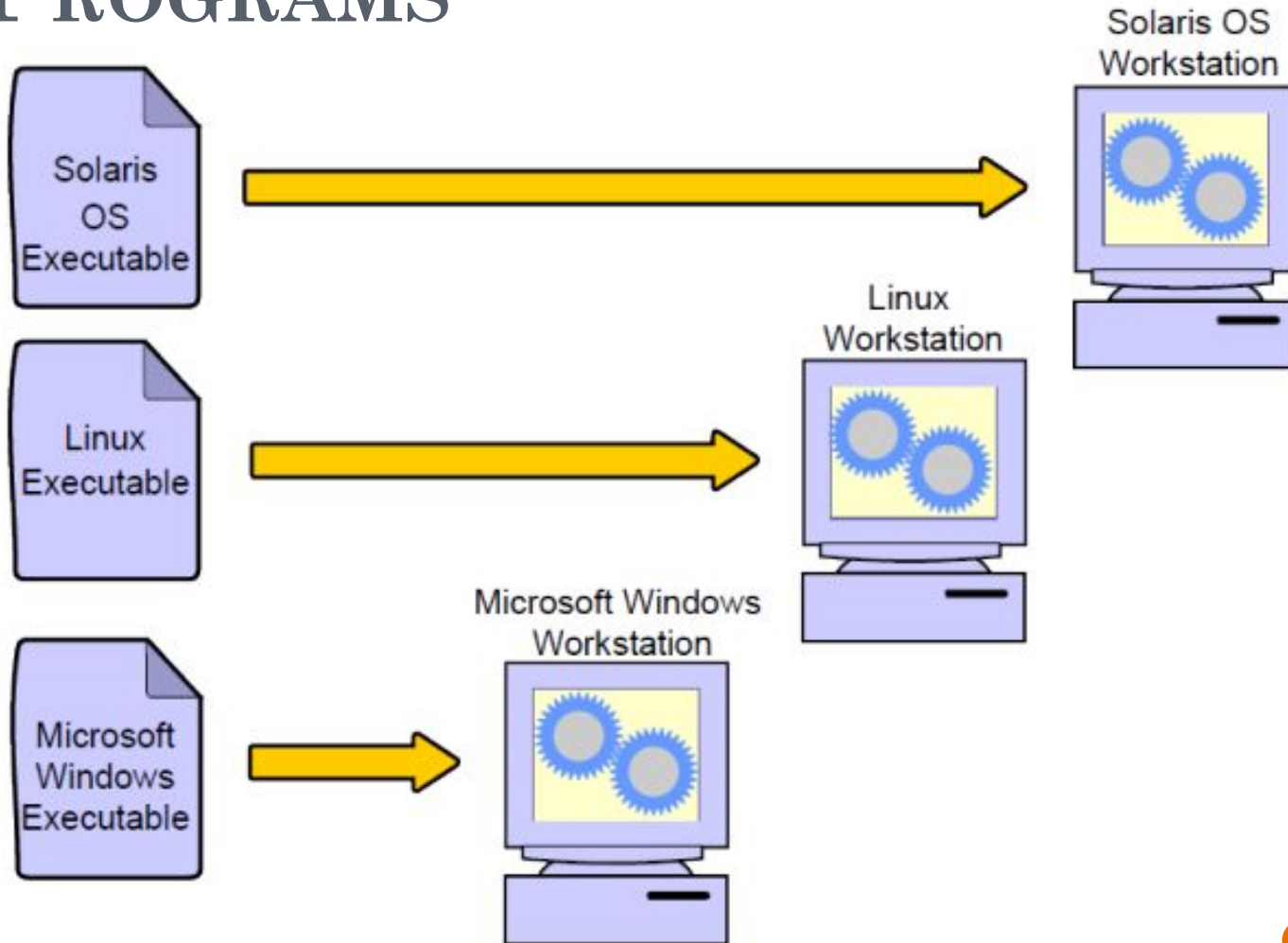
LINKED TO PLATFORM-SPECIFIC LIBRARIES



Source: Oracle Academy



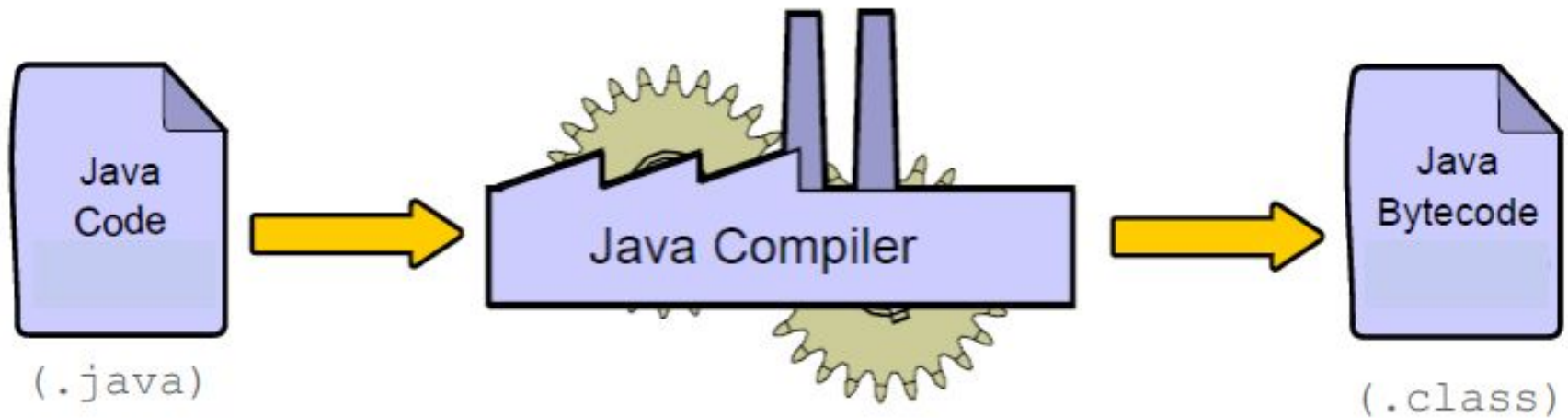
PLATFORM-DEPENDENT PROGRAMS



Source: Oracle Academy



JAVA IS PLATFORM-INDEPENDENT

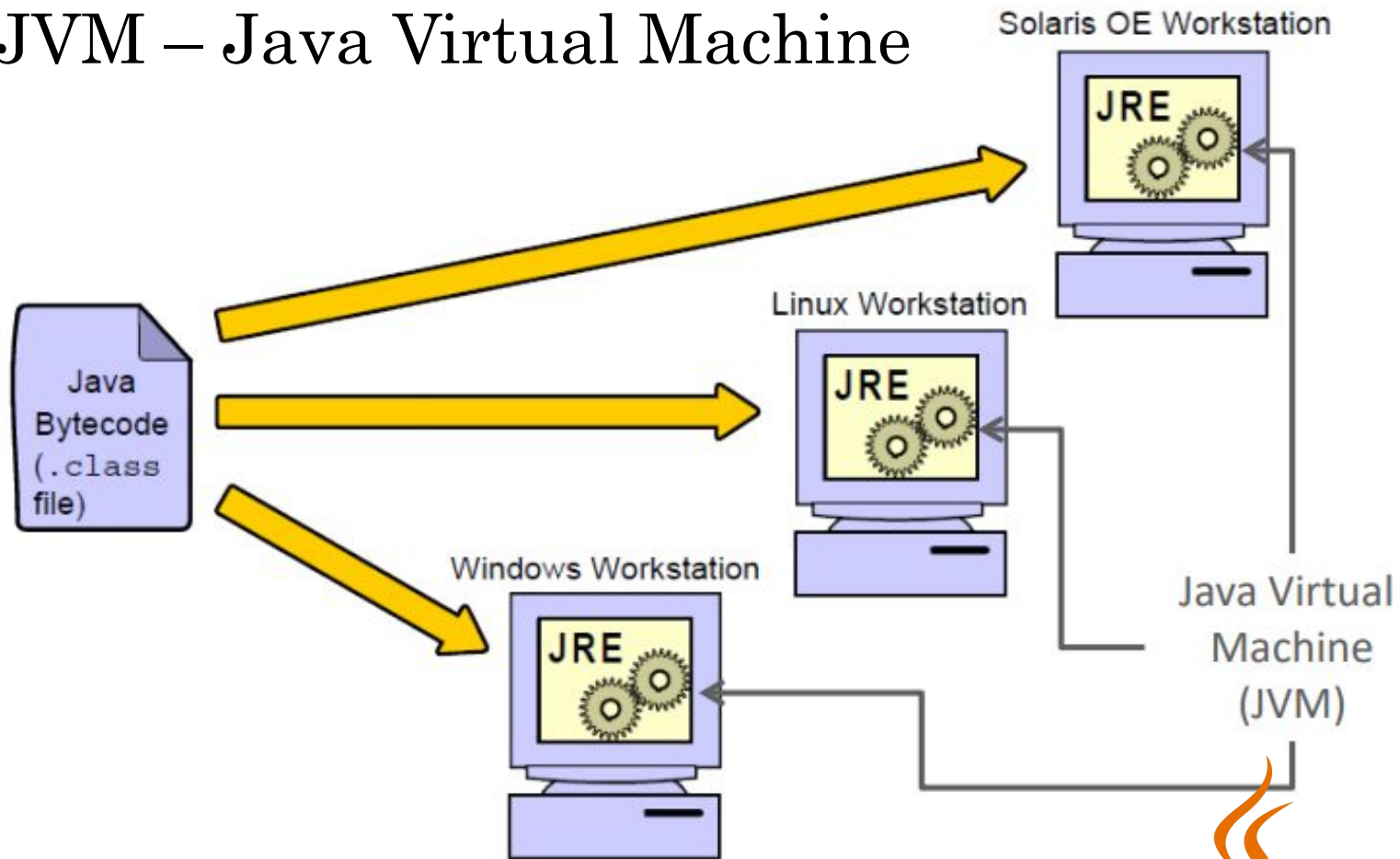


Source: Oracle Academy



JAVA PROGRAMS RUN IN A JVM

□ JVM – Java Virtual Machine



Source: Oracle Academy

JAVA RUNTIME ENVIRONMENT (JRE)

- Includes:
 - The Java Virtual Machine (JVM)
 - Java class libraries
- Purpose:
 - Read bytecode (.class)
 - Run the same bytecode anywhere with a JVM



JAVA DEVELOPMENT KIT (JDK)

- Includes:
 - JRE –
 - Java Compiler
 - Additional tools
- Purpose:
 - Compile bytecode
(.java □ .class)



INTEGRATED DEVELOPMENT ENVIRONMENT (IDE)

□ Purpose:

- Provide a sophisticated text editor
- Offer assistance debugging code
- Manage projects
- Write source code (.java)

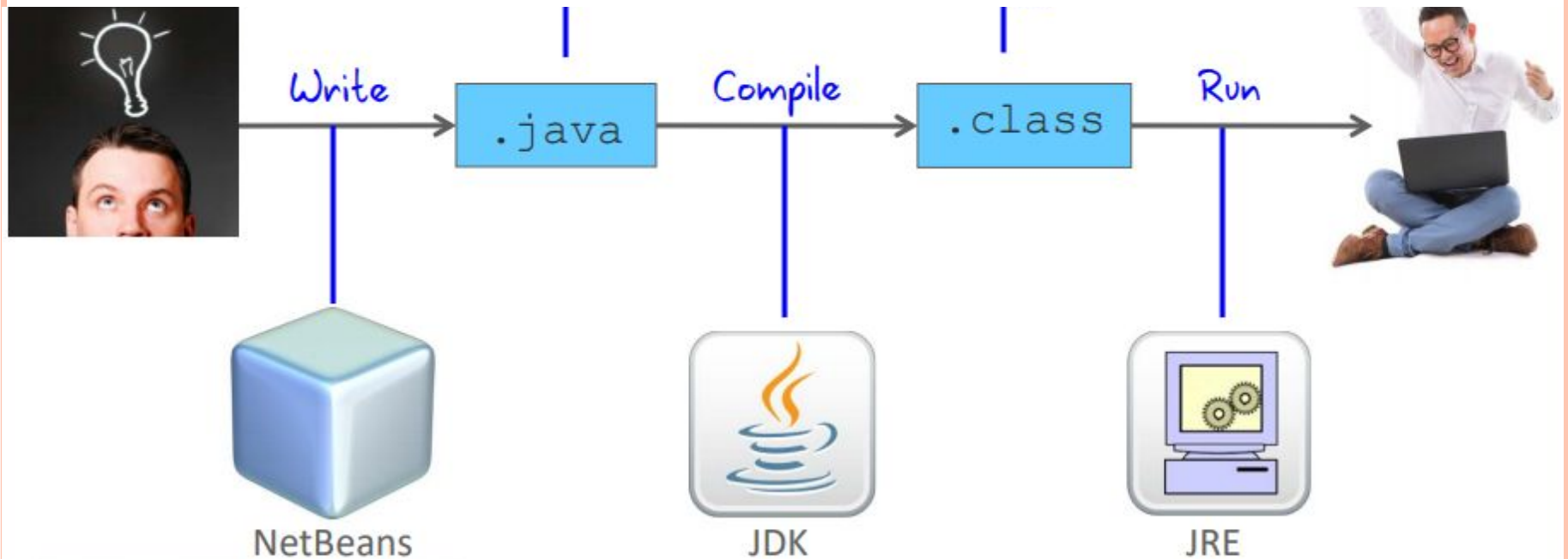
□ Examples:

- NetBeans
- Greenfoot and BlueJ
- Alice

Source: Oracle Academy



COMPILING AND RUNNING A JAVA PROGRAM



Source: Oracle Academy



JAVA PROGRAM BASIC STRUCTURE

42



```
package com.javafiledemo;
```

Package
Declaration

```
import java.util.*;
```

```
import java.io.*;
```

Import
Statements

```
class Class1{  
}
```

```
class Class2{  
}
```

```
class ClassN{  
}
```

Class(es)

- Any Java programs can contain any number of classes
- ***At most one class*** can be declared as ***public***
- If there is a public class, the name of the source file of the program and the name of the public class must be ***match or the same***
- If there ***is no public class*** then we can use ***any name*** for java source file




```
package com.javafiledemo

public class Class1{
}

class Class2{
}

class ClassN{
}
```

□ Program Filename: *Class1.java*

```
package com.javafiledemo
```

```
class Class1{  
}
```

```
class Class2{  
}
```

```
class ClassN{  
}
```

- Program Filename: *Class1.java*,
Class2.java, *ClassN.java*, or
JunAr.java

Source: Java Deep Learning (Awwalsoft)

```
package com.javafiledemo

public class Class1{
}

Public class Class2{
}

class ClassN{
}
```

□ Program Filename: *Class2.java*

- To improves readability and understandability of the code:
 - Take only ***one class for source file*** and name of the program (file) must ***be same*** as class name

Program Filename: ***Class1.java***

```
package com.javafiledemo  
  
public class Class1{  
  
}
```

Source: ***Java Deep Learning (Awwalsoft)***



KEEP IN MIND:

Class Names

- Should be in a *Pascal Case* format (*First letter should be in Upper Case. If several words are used to form a name, each inner word's first letter should be in Upper Case.*)
- Example:
 - MyFirstJavaProgram
 - HelloWorld
 - Rectangle

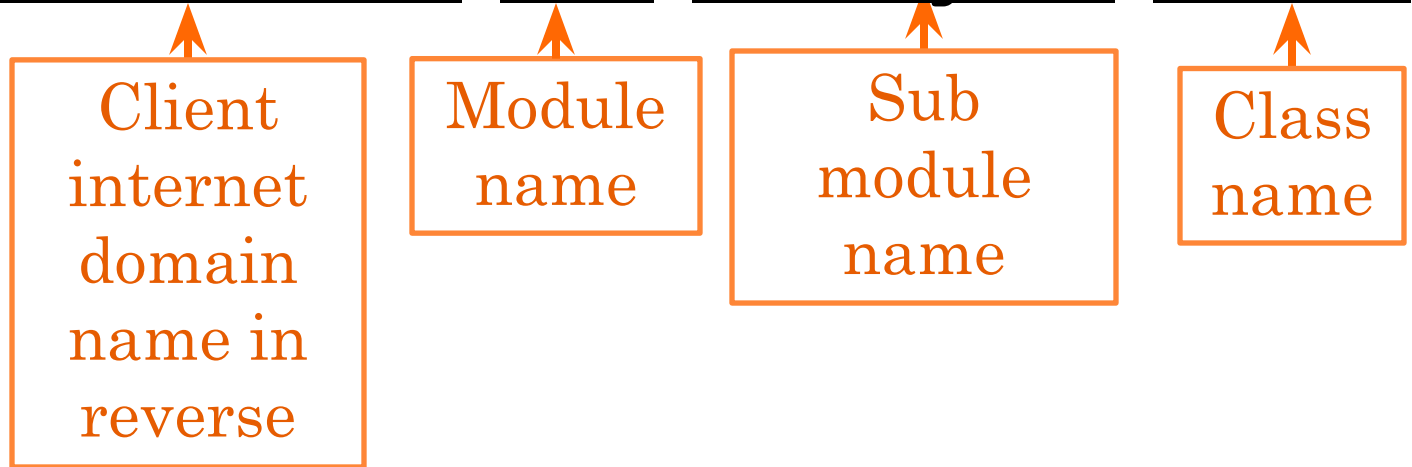
THE PACKAGE STATEMENT

- It is an encapsulation mechanism to group related classes and interfaces into a single module.
- The main objectives of packages are:
 1. To resolve name conflicts
 2. To improve modularity of the application
 3. To provide security



- Universally accepted naming convention for packages that is to use internet domain name in reverse:
- Example:

package com.icicibank.loan.housingloan.Account



```
package myfirstjavaprogram;
```

Package

```
/**
```

```
 * @Title: Program#01
```

```
 * @Description: A program that will display "Hello Java World"
```

```
 * @Author: June Arreb C. Danila
```

```
 * @DateCreated: 6/11/2018
```

```
 */
```

```
public class MyFirstJavaProgram {
```

```
    public static void main(String[] args){
```

```
        //Display a text "Hello World"
```

```
        System.out.println("Hello Java World");
```

```
    } //End of main() method
```

```
} //End of MyFirstJavaProgram class
```

Comment

Statement

Comment

Program
Documentation
(Block Comment)

Class
Block

Required
main()
Method Block

THE MAIN() METHOD

- ❑ The class may contain a required main() method or not
- ❑ But the required main() method is the method find by the JVM when running the program
- ❑ The compiler is not responsible for checking the required main() methods
- ❑ At the runtime, JVM is the responsible for checking the required main() methods

Source: Java Deep Learning (Awwalsoft)



- ❑ If JVM unable to find the required `main()` method the program will get a runtime exception ***NoSuchMethodError: main***
- ❑ The required `main()` method has the following prototype:

To call by JVM
anywhere

W/o existing object JVM
has to call this method

public static void main(String[] args)

The `main()` method won't
return anything to JVM

Command line
arguments



Acceptable changes to main() method syntax:

1. The order of modifiers (public, static) can be change
2. Can declare *string[]* in any acceptable form: *String[] args*, *String []args*, or *String args[]*
3. Can use any valid java identifier instead of *args*
4. Can replace *string[]* with *var-arg* parameter.
Ex: *main(String... args)*
5. main() method can be add the following modifiers: *final*, *synchronized*, *strictfp*



Tell valid or invalid main() methods

1. `public static void main(String args){ }`
2. `public synchronized final strictfp void main(String[] args){ }`
3. `public static void main(String... args){ }`
4. `public static int main(String[] args){ }`
5. `public static synchronized final strictfp void main(String... args){ }`
6. `public static void main(String... args){ }`
7. `public void main(String args[]){ }`
8. `static public void main(String[] args){ }`
9. `public static synchronized void main(String[] a){ }`
10. `static public final main(String []june){ }`

Tell valid or invalid main() methods

1. `public static void main(String args){ } //invalid`
2. `public [static] synchronized final strictfp void main(String[] args){ } //invalid, no static`
3. `public static void Main(String... args){ } //invalid`
4. `public static int main(String[] args){ } //invalid`
5. `public static synchronized final strictfp void main(String... args){ } //valid`
6. `public static void main(String... args){ } //valid`
7. `public [static] void main(String args[]){ } //invalid,`
8. `static public void main(String[] args){ } //valid`
9. `public static synchronized void main(String[] a){ }
//valid`
10. `static public final [void] main(String []june){ }
//invalid`

JAVA COMMENTS

▣ Single-line comment

- Starts with //

▣ Multiline comment

- Begins with /* and ends with */

▣ Documentation comment

- This type of comment is used to produce an HTML file that documents your program
- Begins with a /** and ends with a */

Documentation comment

```
1 package myfirstjavaprogram
2 /**
3  * @Title: Program#01
4  * @Description: A program that will display "Hello Java World"
5  * @Author: June Arreb C. Danila
6  * @DateCreated: 6/11/2018
7  */
```

```
8 public class MyFirstJavaProgram
```

```
9 {
```

```
10
11     public static void main(String[] args) {
```

```
12     {
```

```
13         // TODO code application logic here
```

```
14
15         /* The following statement will command
16            the computer to display "Hello Java World"*/
```

```
17         System.out.println("Hello Java World");
```

```
18     }
```

```
19
20 }
```

Single-line comment

Multiline comment

JAVA PROGRAM BASIC ELEMENTS

60



IDENTIFIERS

- ▣ *Identifier* is a name in Java program
- ▣ Maybe a class name, method name, variable name, and label name

```
class Test
{
    public static void main(String[] args)
    {
        int x = 10;
    }
}
```

RULES TO DEFINED JAVA IDENTIFIERS

- ❑ The only allowed characters are [a - z], [A - Z], [0 - 9], underscore(_) and dollar sign (\$). Using any character will get a compile error.
- ❑ Identifiers are not allowed to start with digit.
- ❑ Identifiers are case sensitive
- ❑ There is no length limit for java identifiers but it is recommended to take not more than 15 lengths
- ❑ Do not use reserved words as identifiers
- ❑ All pre-defined java class names and interface names are allowed to use as identifiers (*but it is not good programming exercise*)

RESERVED WORDS

- Reserved identifiers that are used to associate some functionality or meaning

Reserved Words (53)		
Keywords (50)		Reserved Literals (3) <i>true</i> <i>false</i> <i>null</i>
Used Keywords (48)	Unused Keywords (2) <i>goto</i> <i>const</i>	

▣ *Reserve words for data types (8)*

byte	short	int	long
float	double	char	boolean

▣ *Reserve words for flow control (11)*

if	else	switch	case
default	for	do	while
break	continue	return	

▣ *Keywords for modifiers (11)*

public	private	protected	static
final	abstract	synchronized	native
strictfp	transient	volatile	

▣ *Keywords for exception handling (6)*

try	catch	finally
throw	throws	assert

▣ *Class Related Keywords (6)*

class package import
extends implements interface

▣ *Object Related Keywords (4)*

new super instanceof this

▣ *Void return type keyword:* void

▣ *Reserved Literals:* true, false, null

▣ *Enumerated type keyword:* enum

▣ *Unused Keywords:* goto, const

PRIMITIVE DATA TYPES

- Integer value
 - **byte, short, int, long**
- Floating-point value
 - **float, double**
- Boolean value
 - **boolean**
- Character value
 - **char**

byte

- 8-bit signed 2's complement integer
- value range:
 - **-128** (-2^7) to **127** (2^6)
- default value is **0**

short

- 16-bit signed 2's complement integer.
- value range:
 - **-32,768** (-2^{15}) to **32,767** (2^{14})
- default value is **0**

int

- 32-bit signed two's complement integer.
- value range:
 - **-2,147,483,648** (-2^{31}) to **2,147,483,647** (2^{30})
- default value is **0**

long

- 64-bit signed two's complement integer.
- value range:
 - **-9,223,372,036,854,775,808** (-2^{63}) to **9,223,372,036,854,775,807** (2^{62})
- default value is **0L**

float

- a single-precision 32-bit IEEE 754 floating point
- default value is **0.0F**

double

- a double-precision 64-bit IEEE 754 floating point
- default value is 0.0

boolean

- represents one bit of information.
- possible values: **true & false**
- used for simple flags that track true/false conditions.
- default value is false

char

- a single 16-bit Unicode character.
- range value: `'\u0000'` (or 0) to `'\uffff'` (65,535)
- used to store any character

JAVA LITERALS

- ▣ *Literal* is any constant value which can be assigned to the variable
- ▣ *Java Literals:*
 - *Integral Literals*
 - *Floating Point Literals*
 - *Boolean Literals*
 - *Character Literals*
 - *String Literals*
 - *Binary Literals*

Integral Literals

- Assigned for the integral data types (byte, short, int, long)

- Decimal Literals***: allowed digits *0 to 9*

```
int x = 10; // x = 10
```

- Octal Literals***: allowed digits *0 to 7* and prefixed with *0*

```
int x = 010; // x = 8
```

- Hexadecimal Literals***: allowed digits are *0 to 9* and *A to F* or *a to f* and prefixed with *0x* or *0X*

```
int x = 0x10; // x = 16
```

Floating Point Literals

- Floating point literal is by default *double type*

```
float f = 123.456; //C.E. Loss of precision  
double d = 123.456; //valid
```

- To specify explicitly as *float type* suffixed with *f* or *F*

```
float f1 = 123.456f; // (valid)  
float f2 = 123.456F; // (valid)
```

- To specify explicitly as ***double type*** suffixed with ***d*** or ***D***

```
double d1 = 123.456d; // (valid)
```

```
double d2 = 123.456D; // (valid)
```

- Can specify floating point literal only in decimal form and can't specify in octal and hexadecimal forms

```
double d1 = 123.456; // (valid)
```

```
double d2 = 0123.456; // treated as 123.456
```

```
double d3 = 0x123.456; // C.E. malformed  
// float
```

- Can assigned integral literal directly to the floating point data types and that integral literal can be specified in decimal, octal, or hexadecimal forms

```
double d1 = 0x10; //d1 = 16.0 (valid)
```

```
double d2 = 010; //d2 = 8.0 (valid)
```

```
double d3 = 10; //d3 = 10.0 (valid)
```

- Can't assign floating point literal directly to the integral types

```
int i1 = 10.0f; //C.E. Loss of precision
```

```
int i2 = 10.0d; //C.E. Loss of precision
```

- Can specify floating point literal in exponential form:

```
double x = 7.35E9; //7.35 x 10^9 (valid)
```


Boolean Literals

- The only allowed values for the Boolean type are *true* or *false* (case sensitive)

```
boolean b1 = true; // valid
```

```
boolean b2 = 0; // CE: incompatible types
```

```
boolean b3 = True; // CE: cannot find symbol
```

Char Literals

- Can be represented as single character within single quotes

```
char c1 = 'A'; // valid
```

```
char c2 = a; // CE: cannot find symbol
```

```
char c3 = "a"; // CE: incompatible types
```

- Can specify a char literal as integral literal (decimal, octal, or hexadecimal form) which represents Unicode of the character (0 to 65, 535)

```
char c1 = 97; // c1 = 'a' (valid)
```

```
char c2 = 0xFB; // c2 = '√' (valid)
```

```
char c3 = 65536; // CE: Loss of precision
```

- Can represent a char literal by Unicode representation `'\uXXXX'` (4-digit Hexadecimal number)

```
char c1 = '\u0456' ; // c1 = 'i'
```

- Can assign the char literal to integral and floating point data types

```
int i = 'A' ; // i = 65
```

```
double d = 'a' ; // d = 97.0
```

- Java escape character acts as a char literal

```
char c1 = '\n' ; //new line character
```

```
char c2 = '\t' ; //tab character
```

Escape Character	Description
\t	Tab
\b	Backspace
\n	Newline
\r	Carriage return
\f	Form feed
\'	Single quote character
\"	Double quote character
\\	Backslash character

String Literals

- Any sequence of characters within double quotes

```
String s1 = "JunAr"; //valid
```

```
String s1 = 'a'; //CE: cannot convert  
           //char to string
```

Binary Literals

- Can specify literal value in binary form
- The allowed digits are **0** and **1** and prefixed with **0b** or **0B**

```
int i = 0b11100; //i = 28
```

Usage of Underscore (_) symbol in Numeric Literals

- Underscore (_) symbol is used to separate the digits in numeric literals
- At the time of compilation the '_' symbol will be removed automatically.
- The main purpose of this is to improved readability.

```
int i = 123_456; //valid
```

```
double d = 123_456.7_8_9; //valid
```

JAVA VARIABLE DECLARATION

- ▣ *Variable* is an identifier use to represent a memory space
- ▣ A *variable* is defined by the combination of a *modifier*, a *type*, an *identifier*, and an optional *initializer*

`[modifiers] type identifier [= value]
[, identifier [= value] ...] ;`

```
int x, y, z;
```

```
int operand1 = 2, operand2 = 3;
```

```
static double rad;
```

▣ ***Dynamic Initialization*** allows variables to be initialized dynamically, using any expression valid at the time the variable is declared

```
double a = 2.0, b = 4.0;

/*a_square & b_square
is dynamically initialized*/
double a_square=a*a, b_square=b*b;

// c is dynamically initialized
double c = Math.sqrt(a_square + b_square);
```


JAVA VARIABLE TYPES

- Types of Variable (*Based on value represented*)
 1. Primitive variables
 2. Reference variables

- Types of Variable (*Based on the behavior and position of declaration*)
 1. Block variables
 2. Instance variables
 3. Static variables

Primitive Variables

- Can be used to represent primitive values

```
int x = 10;
```

Reference Variables

- Can be used to refer objects

```
Student s = new Student();
```

Diagram:



<i>Memory</i>	
x	10
c	'A'
d	10.0
b	true

Block Variables

- ▣ *Local Variables, Automatic Variables, Temporary Variables, Stack Variables*
- ▣ *Block variables* are variables that declared inside a method, block, or constructor
- ▣ It will created as part of the block execution in which it is declared and destroyed once that block execution completes
- ▣ The scope of block variable is exactly same as scope of the block in which it is declared

```

public class MyFirstJavaProgram
{
    public static void main(String[] args)
    {
        int sum = 0;
        for(int i = 1; i<=10; i++)
        {
            sum = sum + i;
        }
        double average = sum/i;
        System.out.println("Average: " + average);
    }
}

```

cannot find symbol
symbol: variable i
location: class MyFirstJavaProgram

(Alt-Enter shows hints)

```
public static void main(String[] args)
{
    //Variable args is visible only inside main() method

    //Variable sum is visible only inside main() method
    int sum;
    {
        //Variable num1 & num2 is visible only inside this block
        int num1=5, num2=3;
        System.out.println("The value of num1 is " + num1);
        System.out.println("The value of num2 is " + num2);

        sum=num1+num2;
    }

    System.out.println("The sum of num1 & num2 is " + sum);
}
```

```
public static void main(String[] args)
{
    int x=10; // known to all code within main
    if(x == 10)
    {
        int y = 20; // known only to this block
        // x and y both known here.
        System.out.println("x and y: " + x + " " + y);
        x = y * 2;
    }
    y = 100; // Error! y not known here
    // x is still known here.
    System.out.println("x is " + x);
}
```

Instance Variables

- ▣ *Instance variables* are variables where the value is varied from object to object
- ▣ For every object a separate copy of instance variables will be created
- ▣ It will be created at the time of object creation and destroyed at the time of object destruction
- ▣ The scope of instance variable is exactly same as scope of objects
- ▣ Instance variables will be stored on the *heap* as the part of object

- ❑ **Rule 1:** Instance variables should be *declared with in the class* directly but *outside of any method, block, or constructor*
- ❑ **Rule 2:** Instance variables can be access directly from an instance area but cannot be accessed directly from the static area
- ❑ **Rule 3:** Instance variables can be access from static area by using object reference


```
public class JavaSample
```

```
{
```

```
    int sum; //Rule 1
```

```
    //main() method is static area
```

```
    public static void main()
```

```
    {
```

```
        sum = 0; //violates Rule 2
```

```
        for(int i = 1; i<=10; i++)
```

```
            sum = sum + i;
```

```
    }
```

```
    //displaySum() method is an instance area
```

```
    public void displaySum()
```

```
    {
```

```
        System.out.println("Sum: " + sum); //Rule 2
```

```
    }
```

```
}
```

non-static variable sum cannot be referenced from a static context

(Alt-Enter shows hints)

```
public class JavaSample
{
    int sum; //Rule 1
    //main() method is static area
    public static void main(String[] args)
    {
        //Rule 3: Use object reference
        JavaSample sample = new JavaSample();
        sample.sum = 0;
        for(int i = 1; i<=10; i++)
            sample.sum = sample.sum + i;
        sample.displaySum();
    }
    //displaySum() method is an instance area
    public void displaySum()
    {
        System.out.println("Sum: " + sum); //Rule 2
    }
}
```

Static Variables

▣ *Class variables*

- ▣ *Static variable* is a class level variable where the value is not varied from object to object.
- ▣ Only one copy of static variables will be created for the entire class and shared by every object of that class
- ▣ Static variables will be created at the time of class loading and destroyed at the time of class unloading
- ▣ The scope of static variable is exactly the same as the scope of the *.class* file
- ▣ Static variables will be stored in *method area*

- ❑ **Rule 1:** Static variables should be *declared with in the class* directly but *outside of any method, block, or constructor* and be declared with **static** modifier
- ❑ **Rule 2:** Static variables can be access from both instance and static area directly
- ❑ **Rule 3:** Static variables can access either by class name or by object reference (*usage of class name is recommended*)
- ❑ **Rule 4:** When accessing within the same class, it is not required to use class name to access the static variables directly

```
public class JavaSample
{
    static int sum; //Rule 1
    //main() method is static area
    public static void main(String[] args)
    {
        //Rule 3: Using object reference
        JavaSample sample = new JavaSample();
        sample.sum = 0;
        for(int i = 1; i<=10; i++)
            //Rule 3: Using class name
            JavaSample.sum = sum/*Rule 2 & 4*/ + i;
        sample.displaySum();
    }
    //displaySum() method is an instance area
    public void displaySum()
    {
        System.out.println("Sum: " + sum); //Rule 2 & 4
    }
}
```

```
public class JavaSample
{
    //main() method is static area
    public static void main(String[] args)
    {
        Sum.sum = 0;
        for(int i=1; i<=10; i++)
            Sum.sum = Sum.sum + i;

        System.out.println("Sum: " + sum);
    }
}
```

cannot find symbol
symbol: variable sum
location: class JavaSample

(Alt-Enter shows hints)

```
class Sum
{
    static int sum;
}
```

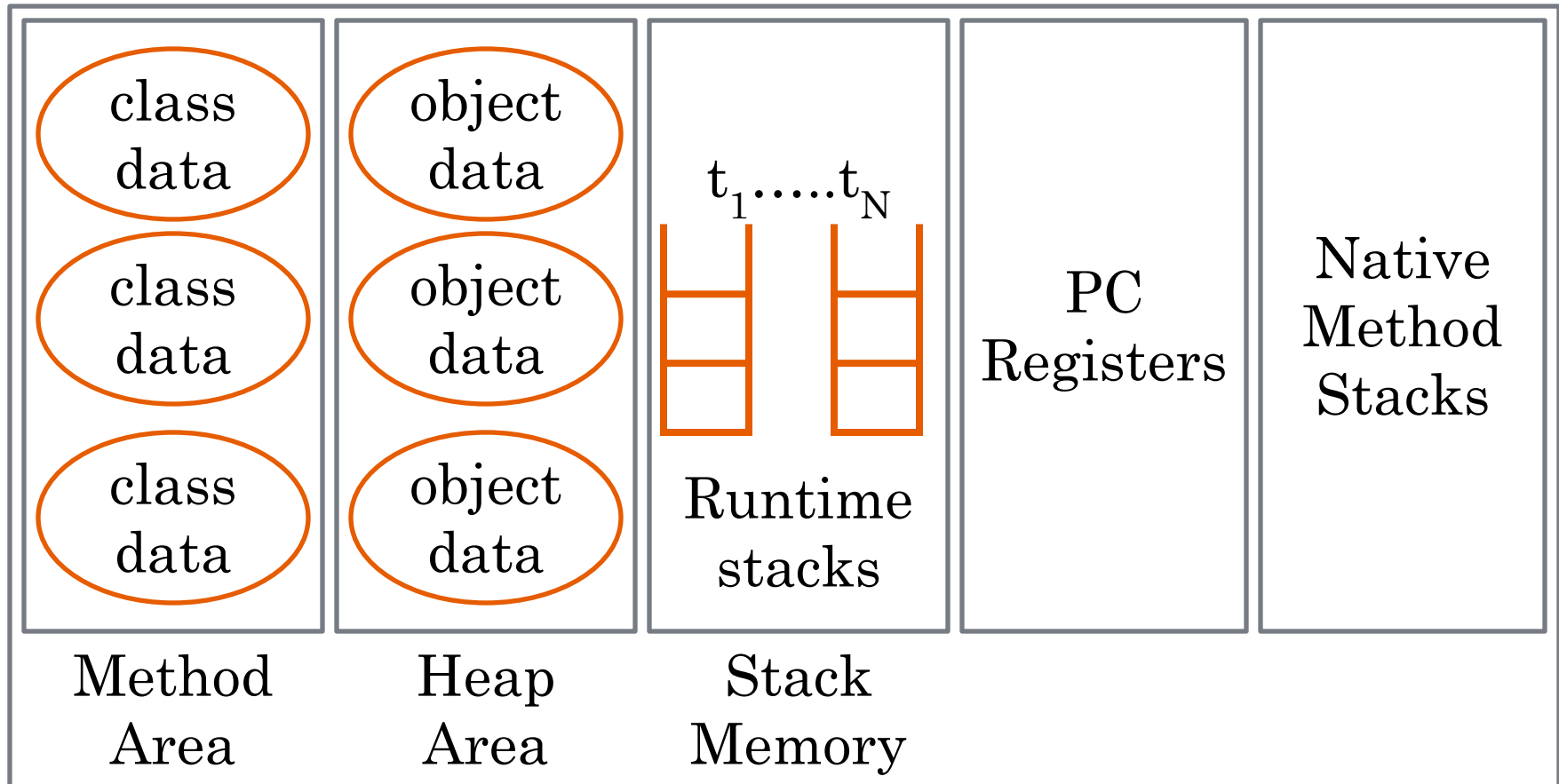


```
public class JavaSample
{
    //main() method is static area
    public static void main(String[] args)
    {
        Sum.sum = 0;
        for(int i=1; i<=10; i++)
            Sum.sum = Sum.sum + i;

        System.out.println("Sum: " + Sum.sum);
    }
}

class Sum
{
    static int sum;
}
```

JVM MEMORY AREAS



- ❑ Class level binary data (*static variables*, static blocks, references) will stored in **method area**
- ❑ Objects and corresponding *instance variables* will stored in **heap area**
- ❑ For every method the JVM will create a **runtime stack**, all method calls perform by that thread and corresponding *local variables* will be stored in that stack. Every entry in stack is called *stack frame* or *action record*
- ❑ The instruction which has to execute next will be stored in corresponding **PC registers**
- ❑ Native methods invocations will be stored in **native method stacks**

JAVA FINAL VARIABLES

- ❑ *Final variable* is the *constant variable* in java
- ❑ Java doesn't have built-in support for constants, but the variable modifiers *final* can be used to effectively create one
- ❑ Creating a final variable format:
`[modifiers] final type identifier = value;`

```
public class MyFirstJavaProgram
{
    //declare a static constant PI
    static final double PI = 3.14;
    public static void main(String[] args)
    {
        //declare constant inside the method
        final int constant_value=2;

        double radius = 3.0;
        double circumference = constant_value * PI * radius;
        System.out.println("Circumference = " + circumference);
    }
}
```

JAVA OPERATORS

String Concatenation Operator	Increment & Decrement Operators
Arithmetic Operators	Relational Operators
Equality Operators	instanceof Operator
Bitwise Operators	Logical Operators
Type-cast Operators	Assignment Operators
Conditional Operator	new Operator
[] Operator	

String Concatenation Operator

- The only overloaded operator in java is '+'
- Acts as string concatenation operator if one argument is String type
- Acts as arithmetic addition if both arguments are number type

```
String s;
```

```
s = "junar" + '4' + 5 + 6.0; //s=junar456.0
```

```
s = '4' + 5 + 6.0 + "junar"; //s=63.0junar
```

```
s = '4' + "junar" + 5 + 6.0); //s=4junar56
```

```
s = '4' + 5 + 6.0; //CE: double cannot  
//converted to String
```

Increment & Decrement Operators

Increment(++)	
pre-increment <i>Ex: y = <u>++x</u>;</i>	post-increment <i>Ex: y = <u>x++</u>;</i>
Decrement(--)	
pre-decrement <i>Ex: y = <u>--x</u>;</i>	post-decrement <i>Ex: y = <u>x--</u>;</i>

```
public static void main(String[] args)
{
    int x=10;
    int y = ++x;
    System.out.println("x = " + x + ", y = " + y);
    y = x++;
    System.out.println("x = " + x + ", y = " + y);
    y = --x;
    System.out.println("x = " + x + ", y = " + y);
    y = x--;
    System.out.println("x = " + x + ", y = " + y);
}
```

 **Output - JavaSample (run)** X




run:
x = 11, y = 11
x = 12, y = 11
x = 11, y = 11
x = 10, y = 11
BUILD SUCCESSFUL (total time: 0 seconds)

- ❑ **Rule 1:** Increment & decrement operators can apply only for variables but not for constant values

```
int x = 4;
```

```
int y = ++x;
```

```
y = ++4; //CE: Unexpected type
```

- ❑ **Rule 2:** Cannot perform nesting of increment or decrement operator

```
int x = 4;
```

```
int y = --(x++) ; //CE:
```

```
//Unexpected type
```

- ❑ **Rule 3:** Cannot apply increment or decrement operators to the final variables

```
final int x = 4;
```

```
x++; //CE: Cannot assign value to  
      //final variable 'x'
```

- ❑ **Rule 4:** Increment or decrement operators can apply to every primitive types except for *boolean*

```
int x = 10;
```

```
System.out.println(x++) ; //output:10
```

```
char ch = 'b' ;
```

```
System.out.println(--ch) ; //output:a
```

```
double d = 10.5;
```

```
System.out.println(++d) ; //output:11.5
```

```
boolean b = true
```

```
System.out.println(b++) ; //CE
```

//can't be applied to boolean

Arithmetic Operators

- `*` , `/` , `%` , `+` , `-`
- Applying any Arithmetic operation b/w 2 variables (a & b), the result type is always ***max(int, type of a, type of b)***
- Arithmetic operators can apply to every primitive type except boolean

Examples:

`byte + byte = int`

`byte + short = int`

`short + short = int`

`short + long = long`

`long + int = long`

`double + float = double`

`int + double = double`

`char + char = int`

`char + int = int`

`char + double = double`

Arithmetic Exception

```
graph LR; A[Arithmetic Exception] --> B["(1) It is a Runtime Exception but not compile error"]; A --> C["(2) Possible only in integral arithmetic but not in float arithmetic"]; A --> D["(3) The only operators which causes Arithmetic Exception are /, %"]
```

(1) It is a Runtime Exception but not compile error

(2) Possible only in integral arithmetic but not in float arithmetic

(3) The only operators which causes Arithmetic Exception are $/$, $\%$

Relational Operators

□ `< , <= , > , >=`

□ *Comparison operators*

□ The result is always boolean type

□ **Rule 1:** Relational operators can apply to every primitive type except boolean

```
System.out.println(10 < 10.5) ; //true
```

```
System.out.println('a' > 90.5) ; //false
```

```
System.out.println('b' >= 'a') ; //true
```

```
System.out.println(true > false) ; //CE
```

//can't be applied to boolean, boolean

- ❑ **Rule 2:** Relational operators cannot be apply to object types

```
System.out.println("junar" < "JunAr"); //CE  
//can't be applied to java.lang.String,  
//java.lang.String
```

- ❑ **Rule 3:** Nesting of relational operators is not allowed

```
System.out.println(10 > 20 <= 30); //CE  
//can't be applied to boolean, int
```

Equality Operators

- `==` , `!=`
- The result is always boolean type
- **Rule 1:** Relational operators can apply to every primitive type including boolean type also

```
System.out.println(10 == 20); //false
```

```
System.out.println('a' != 97.0); //false
```

```
System.out.println(20 == 20.0); //true
```

```
System.out.println(true != false); //true
```


- ❑ **Rule 2:** Relational operators can apply to object types also. It is use for reference-comparison or address comparison

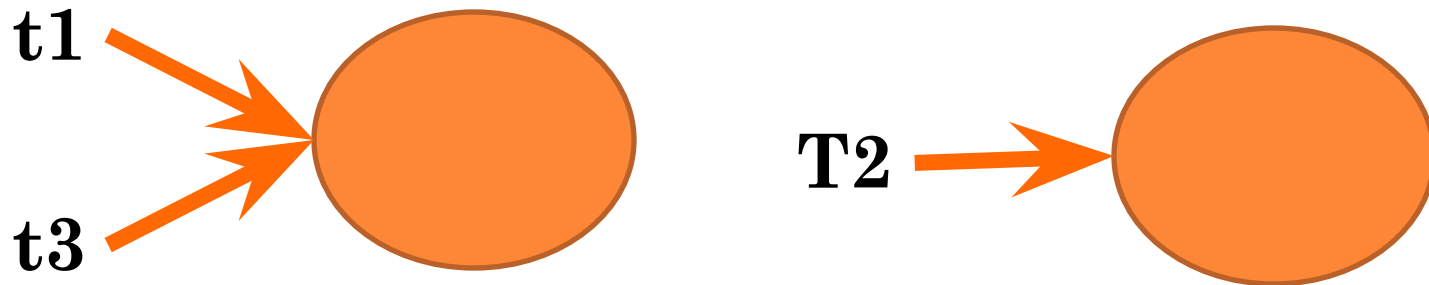
```
Thread t1 = new Thread();
```

```
Thread t2 = new Thread();
```

```
Thread t3 = t1;
```

```
System.out.println(t1 == t2); //false
```

```
System.out.println(t1 == t3); //true
```



- ❑ **Rule 3:** To use the equality operators b/w object type compulsory these should be some relation b/w argument types (child-to-parent, parent-to-child), otherwise it will get Compile error incompatible types

```
Thread t = new Thread();
```

```
Object o = new Object();
```

```
String s = new String("JunAr");
```

```
System.out.println(t == o); //false
```

```
System.out.println(o == s); //false
```

```
System.out.println(s == t); //CE
```

```
//incompatible types: java.lang.String
```

```
//and java.lang.Thread
```

instanceof Operator

- The *instanceof* operator is use to check whether the given object is particular type or not

O instanceof C

- O is object reference, C is class name or interface name

```
Thread t = new Thread();  
System.out.println(t instanceof Thread);  
//true  
System.out.println(t instanceof Object);  
//true  
System.out.println(t instanceof Runnable);  
//true
```

- ❑ **Rule 1:** To use instanceof operator compulsory there should be some relation between argument types (*child-to-parent*, *parent-to-child*, or *same type*) otherwise it will get a compile error

```
Thread t = new Thread();  
System.out.println(t instanceof String);  
//CE: incompatible types
```

- ❑ **Rule 2:** Checking the parent object is child type or not by using instanceof operator will get *false*

```
Object o1 = new Object();  
System.out.println(o1 instanceof String);  
//false
```

```
Object o2 = new String("junar");  
System.out.println(o2 instanceof String);  
//true
```

- ❑ **Rule 3:** The *null* instance of any class/interface will always return false

```
System.out.println(null instanceof  
String); //false
```

Bitwise Operators

□ $\sim$, $\&$, $|$, $\wedge$,

□ **Rule 1:** Bitwise operators (*except for $\sim$ (bitwise compliment)*) can apply to boolean type and the result is boolean

- **& (Bitwise-AND):** the result is true if both arguments are true
- **| (Bitwise-OR):** the result is true if at least one argument is true
- **$\wedge$ (Bitwise-XOR):** the result is true if both are different arguments

```
System.out.println(true & false); //false
```

```
System.out.println(true | false); //true
```

```
System.out.println(true ^ false); //true
```

- ❑ **Rule 2:** Bitwise operator can apply to integral types also and the result is integral types

```
System.out.println(4 & 5); //4
```

```
//100 & 101 => 100 => 4
```

```
System.out.println(4 | 5); //5
```

```
//100 | 101 => 101 => 5
```

```
System.out.println(4 ^ 5); //1
```

```
//100 & 101 => 001 => 1
```

```
System.out.println(~5); //-6
```

Logical Operators

□ `! , && , ||`

□ Short Circuit Operators

□ Rule 1: Logical operators can apply only to boolean type

- **! (Logical-NOT)**: The result is the negation of the boolean arguments
- **&& (Logical-AND)**: The result is *false* if there is a boolean argument that is *false*
- **|| (Logical-OR)**: The result is *true* if there is a boolean argument that is *true*

```
System.out.println(!true) ; //false
```

```
System.out.println(false && true) ; //false
```

```
System.out.println(true || false) ; //true
```


Assignment Operators

- 3 types of assignment operators: Simple, Chained, & Compound assignment

- *Simple Assignment:*

```
int x = 10;
```

- *Chained Assignment:*

```
int a, b, c;
```

```
a = b = c = 20; // a=20, b=20, c=20
```

```
int x, y;
```

```
int z = x = y = 50; // z=50, x=50, y=50
```

```
int a = b = 30; // CE: cannot find symbol b
```

▣ ***Compound Assignment*** is mixed assignment operator with other operator

```
int x = 10;
```

```
a += 40; //a = 50
```

Possible compound assignment operators in java

▣ += , -= , *= , /= , % =

▣ &= , |= , ^=

▣ >>= , >>>= , <<=

Conditional Operator (?:)

- A ternary operator of java
- Format: **(bool_expr) ?value1:value2;**
 - If the bool_expr is true, value1 will be used
 - If the bool_expr is false, value2 will be used

```
int a = 10, b = 20;
```

```
int x = (a > b) ? 30 : 40; //x = 40
```

```
x = (a < b) ? x - b : x + a; //x = 20
```

```
x = (a > b) ? ((a%2 == 1) ? 20 : 10) :  
        ((b%2) == 0) ? 15 : 30); //x=15
```

** The problem is poor readability of the code*

new Operator

- Use to create an object

```
String s = new String("JunAr");
```

[] Operator

- Use to declare under construct/create an array

```
int[] x = new int[5];
```

Type-cast Operators

- Use to explicitly change the type of value
- Format: **(type) value**

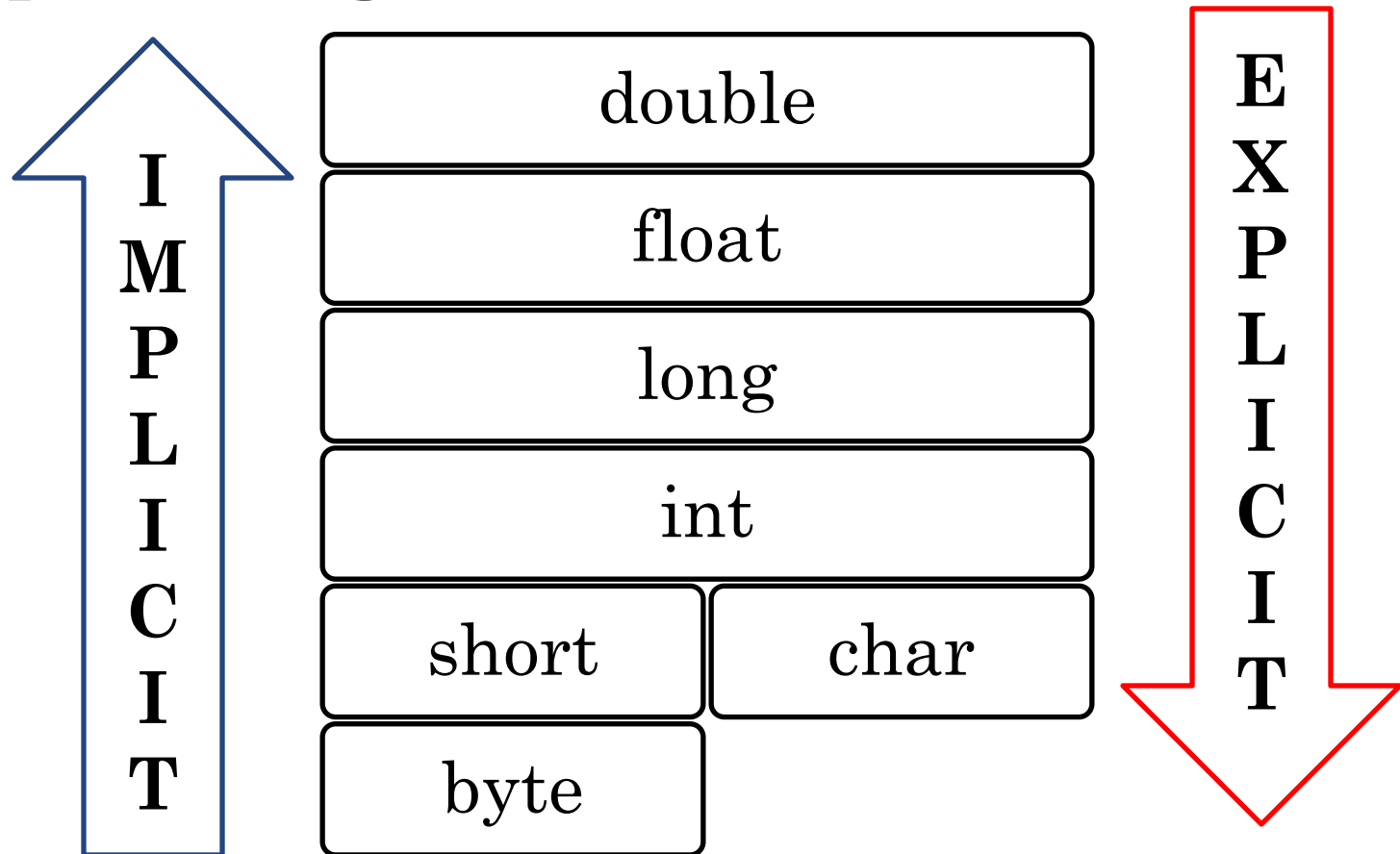
```
int x = (int)2.5; // x = 2
```

JAVA OPERATORS PRECEDENCE

1. Unary operators: `[]` , `x++` , `x--` , `++x` , `--x` , `~` , `new` , `type-cast`
1. Arithmetic operators: `*` , `/` , `%` , `+` , `-`
2. Shift operators: `>>` , `>>>` , `<<`
3. Comparison operators: `<` , `<=` , `>` , `>=` , `instanceof`
4. Equality operators: `==` , `!=`
5. Bitwise operators: `&` , `^` , `|`
6. Logical operators: `&&` , `||`
7. Conditional operator: `?:`
8. Assignment operator: `=` , `*=` , `/=` , `%=` , `+=` , `-=` , `&=` , `|=` , `^=` , `>>=` , `>>>=` , `<<=`

JAVA TYPE-CASTING

- 2 Types of type-casting: *Implicit & Explicit type-casting*



**I
M
P
L
I
C
I
T**

**E
X
P
L
I
C
I
T**

Implicit Type-casting

▣ *Widening, Upcasting, Promotion*

- ▣ The compiler is responsible to perform this type casting
- ▣ When assigning smaller data type to bigger data type variable then implicit type-cast will be performed
- ▣ There is no lose of information in this typecasting

```
int x = 'a' ; // x = 97
```

```
double d = x; // d = 97.0
```


Explicit Type-casting

▣ *Narrowing, down-casting, Demotion*

- ▣ The programmer is responsible to perform this type casting
- ▣ When assigning bigger data type to smaller data type variable then explicit type-casting is required
- ▣ There is a possible lose of information in this typecasting

```
int x = (int)97.0; //x = 97
```

```
char c = (char)x; //c = 'a'
```

```
byte b = x; //CE: Possible loss of  
//precision, byte b = (byte)x
```

- Rule 1: When assigning bigger data type value to a smaller data type value variable by explicit type-casting, the most significant bit will be lost

```
int x = 150;
```

```
byte b = (byte)x; // b = -106
```

- Rule 2: When assigning floating value to the integral types by explicit casting, the digits after the decimal point will be lost

```
double d = 786.97;
```

```
int x = (int)d; // x = 786
```

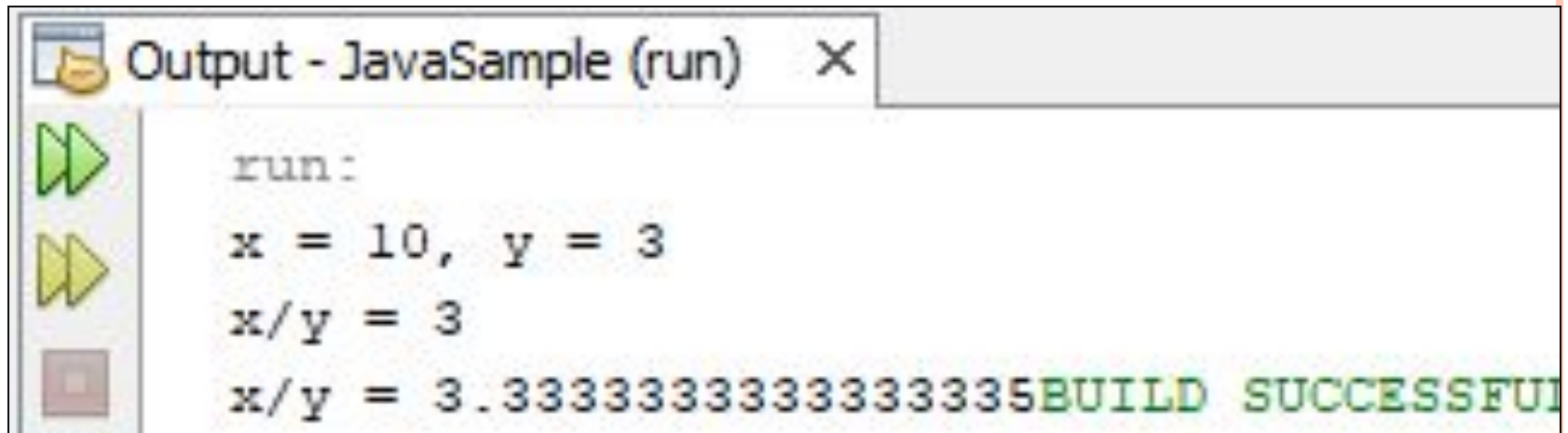
DISPLAY OUTPUT USING SYSTEM.OUT

- Three common methods to display an output:
 - `System.out.print(args);`
 - `System.out.println(args);`
 - `System.out.printf(args);`
- The ***args*** can be a primitive data type value or reference (object) type value

System.out.print() Method

- Display the output w/o a newline.

```
int x = 10, y = 3;  
System.out.print("x = ");  
System.out.print(x);  
System.out.print(", y = " + y);  
System.out.print("\nx/y = " + x/y);  
System.out.print("\nx/y = " + (double)x/y);
```

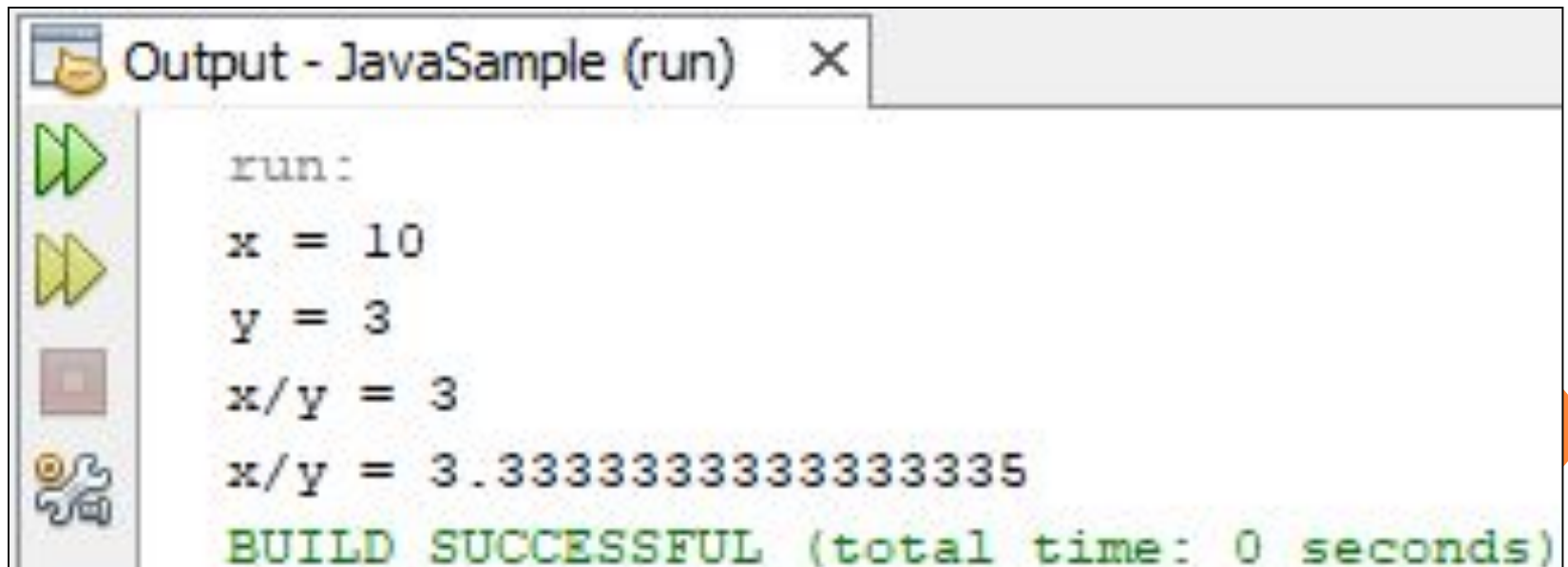


```
run:  
x = 10, y = 3  
x/y = 3  
x/y = 3.33333333333333333335  
BUILD SUCCESSFUL
```

System.out.println() Method

- Display the output a newline at the end.

```
int x = 10, y = 3;  
System.out.println("x = " + x);  
System.out.println("y = " + y);  
System.out.println("x/y = " + x/y);  
System.out.println("x/y = " + (double)x/y);
```



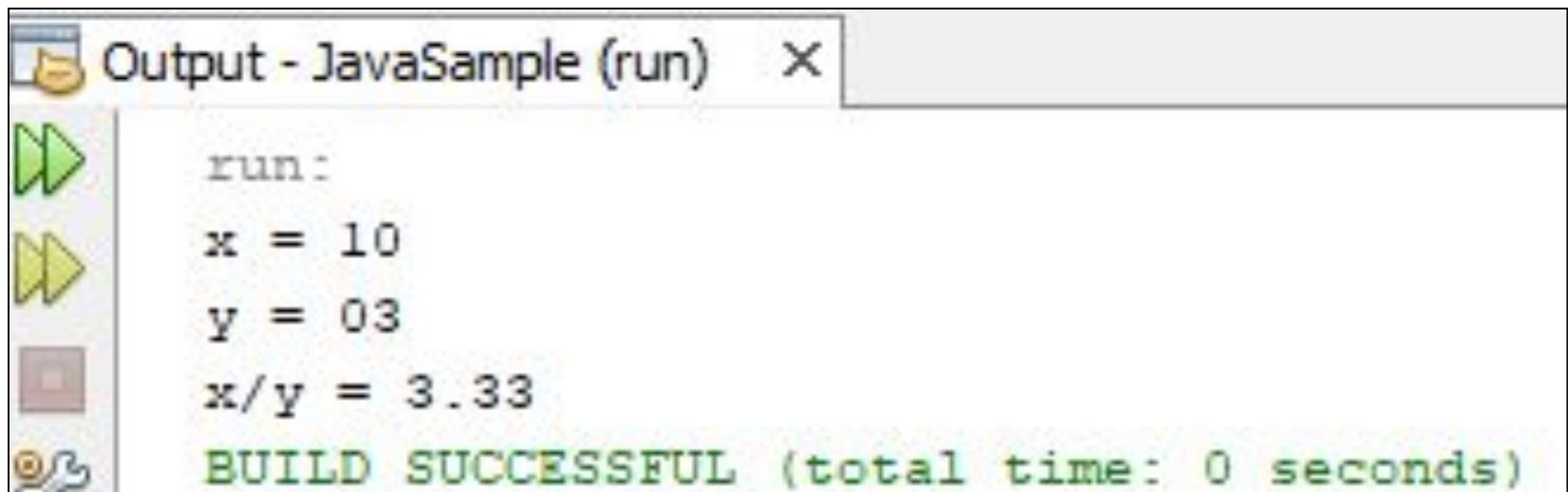
The screenshot shows an IDE's Output window titled "Output - JavaSample (run)". The window contains the following text:

```
run:  
x = 10  
y = 3  
x/y = 3  
x/y = 3.3333333333333333335  
BUILD SUCCESSFUL (total time: 0 seconds)
```

System.out.printf() Method

- Display the output using a placeholder

```
int x = 10, y = 3;  
double z = (double)x/y;  
System.out.printf("x = %02d\n", x);  
System.out.printf("y = %02d\n", y);  
System.out.printf("x/y = %.2f\n", z);
```



```
run:  
x = 10  
y = 03  
x/y = 3.33  
BUILD SUCCESSFUL (total time: 0 seconds)
```



```
int x = 10, y = 3;
double z = (double)x/y;
System.out.print("x=" + x + ", y=" +
                y + ", z=" + z + "\n" );
System.out.println("x=" + x + ", y=" +
                  y + ", z=" + z);
System.out.printf("x=%02d, y=%02d, z=%.2f\n",
                  x, y, z);
```

 Output - JavaSample (run) X
 run:
 x=10, y=3, z=3.33333333333333333335
 x=10, y=3, z=3.33333333333333333335
 x=10, y=03, z=3.33
BUILD SUCCESSFUL (total time: 0 seconds)

USING JAVA

PRE-DEFINED CLASSES

- ❑ Java SDK contains a pre-defined classes and interfaces that can be used in developing a java program
- ❑ The classes and interfaces are grouped into *packages*
- ❑ Each classes in Java SDK has an *API description*
- ❑ *API description* is documentation of a class that gives all of the information you need to be able to syntactically use the class correctly

□ **API description** consists of:

1. **Field Summary**: lists of the static/instance variable names, descriptions, and their types
2. **Method Summary**: lists of all static/instance methods and their descriptions, return type, and arguments.
3. **Constructor Summary**: list of all available constructors are related to the creation of objects

□ Ways to use the Java pre-defined classes:

- Using **fully qualified name** of the class
- Using **import** statement

Using Fully Qualified Name

```
public static void main(String[] args)
{
    //Class Random is a pre-defined class
    //under java.util package
    Random rand1 = new Random();
}
```

cannot find symbol
symbol: class Random
location: class JavaSample

(Alt-Enter shows hints)

```
public static void main(String[] args)
{
    //Class Random is a pre-defined class
    //under java.util package
    java.util.Random rand1 = new java.util.Random();
}
```

Using Import Statement

- 2 package that are not require to import (*package are available by default to every java program*)
 - java.lang package
 - default package (current working directory)

- 2 Types of Import Statements:
 - Explicit Class Import
 - Implicit Class Import

Explicit Class Import

- This type of import is highly recommended to use because it improves readability of the code
- Best suitable for corporate level where readability is important
- Format:

```
import <packagename>.<ClassName>;
```

```
package javasample;

//Explicit import of Class Random
//under java.util package
import java.util.Random;

public class JavaSample
{
    public static void main(String[] args)
    {
        Random rand1 = new Random();
        Random rand2 = new Random();
    }
}
```

Implicit Class Import

- ❑ This type of import is never recommended to use because it reduces readability of the code
- ❑ Best suitable for self-study where typing is important
- ❑ Using the *wildcard* (*) which means all classes of the package
- ❑ Format:

```
import <packagename>.*;
```

```
package javasample;

//Implicit import of Class Random
//under java.util package
import java.util.*;

public class JavaSample
{
    public static void main(String[] args)
    {
        Random rand1 = new Random();
        Random rand2 = new Random();
    }
}
```

Static Import

- To use the static members of the class, w/o using the class name
- Format:

```
import static
```

```
<packagename>.<ClassName>.
```

```
<static_member>;
```



```
package javasample;

//Static import of System
//under java.lang package
import static java.lang.System.out;
public class JavaSample
{
    public static void main(String[] args)
    {
        out.print("Example of ");
        out.println("Static Import");
    }
}
```

CLASS & OBJECT

- ❑ Classes contains a fields and methods
- ❑ If the fields or methods are ***static member***, use the class name to access those static fields or static methods
- ❑ If the fields or methods are ***instance member***, create an **object** (***instance of a class***) of the class to access those instance fields or instance methods
- ❑ Object is a ***reference variable***.
- ❑ To declare and create an object:

```
ClassName obj_var = new ClassName(<args>);
```

(EXAMPLE OF USING STATIC FIELDS & METHODS OF A CLASS)

THE MATH CLASS

- ❑ Package: *java.lang*
- ❑ API link:
<https://docs.oracle.com/javase/6/docs/api/java/lang/Math.html>
- ❑ All fields and methods in this class are *static*
- ❑ Math class has *2 fields*, which are common mathematical constants
- ❑ Math class has more than *37 methods*, which are common mathematical functions
- ❑ Demo Program: *MathClassDemo.java*

(EXAMPLE OF USING INSTANCE FIELDS & METHODS OF A CLASS)

THE RANDOM CLASS

- ❑ Package: *java.util*
- ❑ API link:
<https://docs.oracle.com/javase/6/docs/api/java/util/Random.html>
- ❑ An instance of Random class is used to generate a stream of *pseudorandom numbers*.
- ❑ Random class has *2 constructors* and more than *10 methods*
- ❑ Demo Program: *RandomClassDemo.java*

USER-INPUT USING

THE SCANNER CLASS

- ❑ Package: *java.util*
- ❑ API link:
<https://docs.oracle.com/javase/7/docs/api/java/util/Scanner.html>
- ❑ A simple text scanner which can parse primitive types and strings using regular expressions
- ❑ A Scanner breaks its input into *tokens* using a delimiter pattern, which by default matches *whitespace*(space, newline, tab character)

- The resulting tokens may then be converted into values of different types using the various *next* methods.
- **nextByte()**
 - Scans the next token of the input as a byte
- **nextShort()**
 - Scans the next token of the input as a short
- **nextInt()**
 - Scans the next token of the input as an int
- **nextLong()**
 - Scans the next token of the input as a long

❑ **nextFloat()**

- Scans the next token of the input as a float

❑ **nextDouble()**

- Scans the next token of the input as a double

❑ **nextBoolean()**

- Scans the next token of the input as boolean

❑ **next().charAt(0)**

- Scans the next token of the input as a String (*next()*) and get the first character(*charAt(0)*)

❑ Demo Program: *ScannerClassDemo.java*

JAVA PROGRAM

BASIC FLOW CONTROL

156



FLOW CONTROL

Flow Control

```
graph TD; A[Flow Control] --> B[Selection Statements]; A --> C[Iterative Statements]; A --> D[Transfer Statements];
```

Selection Statements

- if
- if-else
- if-elseif
- switch

Iterative Statements

- while()
- do-while()
- for()
- for-each()

Transfer Statements

- break
- continue
- return
- try-catch-finally
- assert

if Statement:

```
if (bool)
{
    //if block statements;
}
```

▣ ***bool*** is a boolean value or boolean expression

```
public static void main(String[] args)
{
    out.print("Enter a positive number: ");
    Scanner key = new Scanner(System.in);
    int input = key.nextInt();

    if(input > 0)
    {
        out.println("You enter a valid input.");
    }
}
```

if-else Statement:

```
if (bool)
{
    //action stmts if bool is true
}
else
{
    //action stmts if bool is false
}
```

```
public static void main(String[] args)
{
    out.print("Enter a positive number: ");
    Scanner key = new Scanner(System.in);
    int input = key.nextInt();

    if(input > 0)
    {
        out.println("You enter a valid input.");
    }
    else
    {
        out.println("You enter an invalid input.");
    }
}
```

if-elseif-else Statement:

```
if (bool1)
{
    //action stmts if bool1 is true
}
else if (bool2)
{
    //action stmts if bool2 is true
}
else
{
    //action stmts if
    //bool1 & bool2 is false
}
```

```
public static void main(String[] args)
{

    Random rand = new Random();
    int num = rand.nextInt(201)-100;
    if(num > 0)
    {
        out.println(num + " is a positive number.");
    }
    else if(num < 0)
    {
        out.println(num + " is a negative number.");
    }
    else
    {
        out.println("Zero (" + num + ")");
    }
}
```

switch Statement:

- If there are several options, use switch statement to improve readability

```
switch (arg)
{
    case val_label1:
        //action 1;
    case val_label2:
        //action 2;
    case val_labelN
        //action N;
    default:
        //default action;
}
```


- ❑ **Rule 1:** The switch curly braces are mandatory
- ❑ **Rule 2:** The type of *switch argument* (*arg*) and *case label* (*val_label*) should be a *byte*, *short*, *char*, or *int*
- ❑ **Rule 3:** Case label should be constant value or a final variable
- ❑ **Rule 4:** The *switch argument* and *case label* can be an expressions, but *case label* should be constant expression
- ❑ **Rule 5:** Every case label should be within the range of *switch argument*
- ❑ **Rule 6:** There should be no duplication of case label

```
Random rand = new Random();
int val = rand.nextInt(10) + 1;
final int label2 = 20;

switch(val*10)//Rule 4
{
    //Rule 1
    case val + 10://Rule 3 violation
        out.println("TEN");
    case label2://Rule 3
        out.println("TWENTY");
    case 30://Rule 3
        out.println("THIRTY");
    case label2 + label2://Rule 4
        out.println("FORTY");
    case 20 + 20://Rule 6
        out.println("FORTY");
    case 30 + 20://Rule 4
        out.println("FIFTY");
    default:
        out.println("SIXTY ABOVE");
}
```

Fall-Through Inside the Switch

- In switch statement if any case is matched, from that case onwards all statement will be executed until the end of the switch or *break* statement
- The *break* statement is use to exit in the switch block

```

switch (x)
{
    case 0:
        out.print(0);
    case 1:
        out.print(1);
        break;
    case 2:
        out.print(2);
    case 3:
        out.print(3);
    default:
        out.print(4);
}

```

Value of x	Display Output
x<0	4
x=0	01
x=1	1
x=2	234
x=3	34
x>3	4

default Case:

- With in the switch, there can be no default case or only one default case
- The default case can put anywhere with in the switch block (for coding convention, default case as last case)

```

switch(x)
{
    default:
        out.print(4);
    case 0:
        out.print(0);
    case 1:
        out.print(1);
        break;
    case 2:
        out.print(2);
    case 3:
        out.print(3);
}

```

Value of x	Display Output
x<0	4
x=0	01
x=1	1
x=2	234
x=3	34
x>3	4

while() Loop Statement:

```
while (boolean_args)
{
    //while block statements;
}
```

- Is a statement or block of statements that is repeated as long as the condition is satisfied
- The statements inside the while loop (*while block statements*) are executed as long as the *boolean_arg* evaluates to true

do-while() Loop Statement:

```
do
{
    //do-while block statements

}while( boolean_arg );
```

- Similar to the while-loop
- Statements inside a do-while loop (***do-while block statements***) are executed several times as long as the condition is ***true***
- The main difference between a while and do-while loop is that the statements inside a do-while loop are executed at **least once**

for() Loop Statement:

- Allows execution of the same code a number of times
- Normally use when there is a *predetermined number of iterations*

```
for (initExpr; loopCondition; stepExpr)
{
    //for block statement
}
```

- ▣ ***initExpression*** – declare/initializes the loop variable
- ▣ ***loopCondition*** – a Boolean expression that compares the value of the loop variable to some limit value.
- ▣ ***stepExpr*** – an arithmetic expression that updates the value of the loop variable

Loop Control Statements:

break statement

- Terminates the *loop* or *switch* statement and transfers execution to the statement immediately following the loop or switch

continue statement

- Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating

```
public class LoopBreakAndContinueDemo
{
    public static void main(String[] args)
    {
        for(int i = 1;i<=10;i++)
        {
            if(i%5==0)
            {
                break;
            }
            System.out.print(i + " ");
        }
    }
}
```

TRY
THIS
CODE

```
public class LoopBreakAndContinueDemo
{
    public static void main(String[] args)
    {
        for(int i = 1;i<=10;i++)
        {
            if(i%5==0)
            {
                continue;
            }
            System.out.print(i + " ");
        }
    }
}
```

TRY
THIS
CODE

CHAPTER 1B

JAVA STRING AND WRAPPER CLASSES

178



THE JAVA STRING

- ▣ *String* is a sequence of character (i.e. “Hello World”)
- ▣ In java, *string* is treated as an object not an array of characters
- ▣ In java, *string* is an object of *java.lang.String* class that represents sequence of char values
- ▣ *String object* is *immutable object* (*the content cannot be modified once it is created*)

String is Immutable

Sample code 1:

//creating a string object

```
String s = "Hello";
```

//appends " World" string at the end

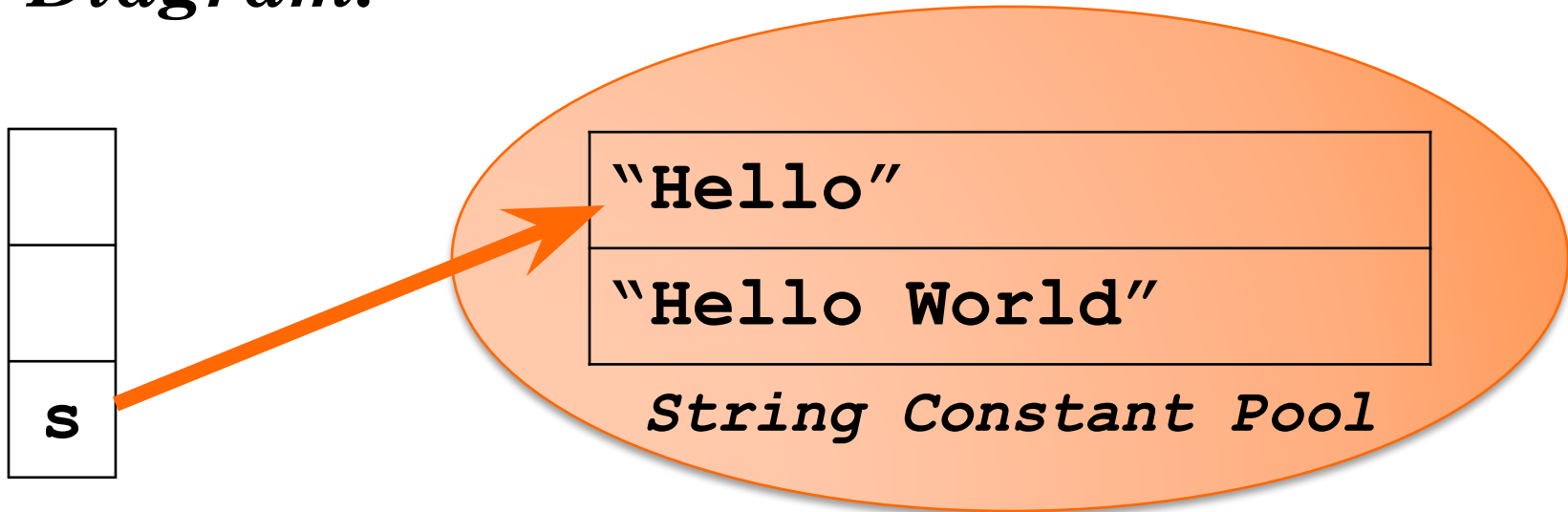
```
s.concat(" World");
```

```
System.out.println(s);
```

Output:

Hello

Diagram:



Sample code 2:

//creating a string object

```
String s = "Hello";
```

//appends " World" string at the end

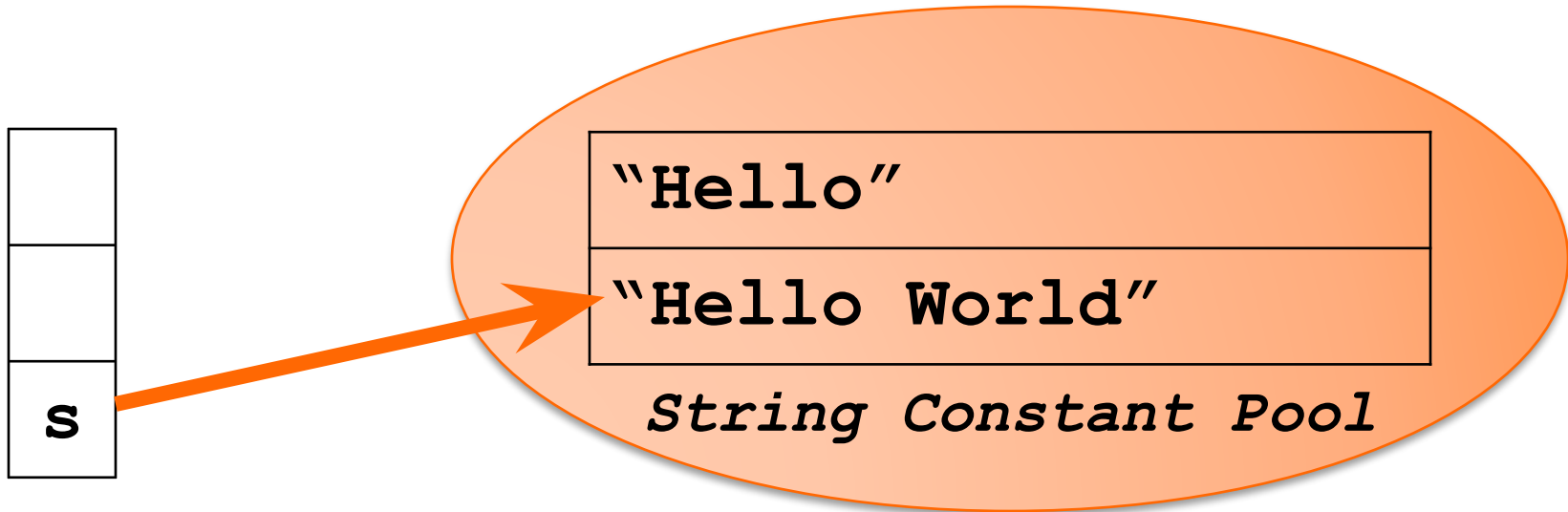
```
s = s.concat(" World");
```

```
System.out.println(s);
```

Output:

Hello World

Diagram:



CREATING A STRING

A String can be constructed by either:

- ❑ directly assigning a *string literal* to a String reference (*Implicit Construction*)
 - *just like a primitive data type*
- ❑ via the *new* operator and any of the *11 constructors* (see the api documentation: <https://docs.oracle.com/javase/8/docs/api/java/lang/String.html>), similar to any other classes (*Explicit Construction*)
 - *not commonly-used and is not recommended*

Creating a String:

Assigning String Literal

□ Sample String Object Instantiation:

//Declaration of String object variable

String str1;*//the value of string
//object is null*

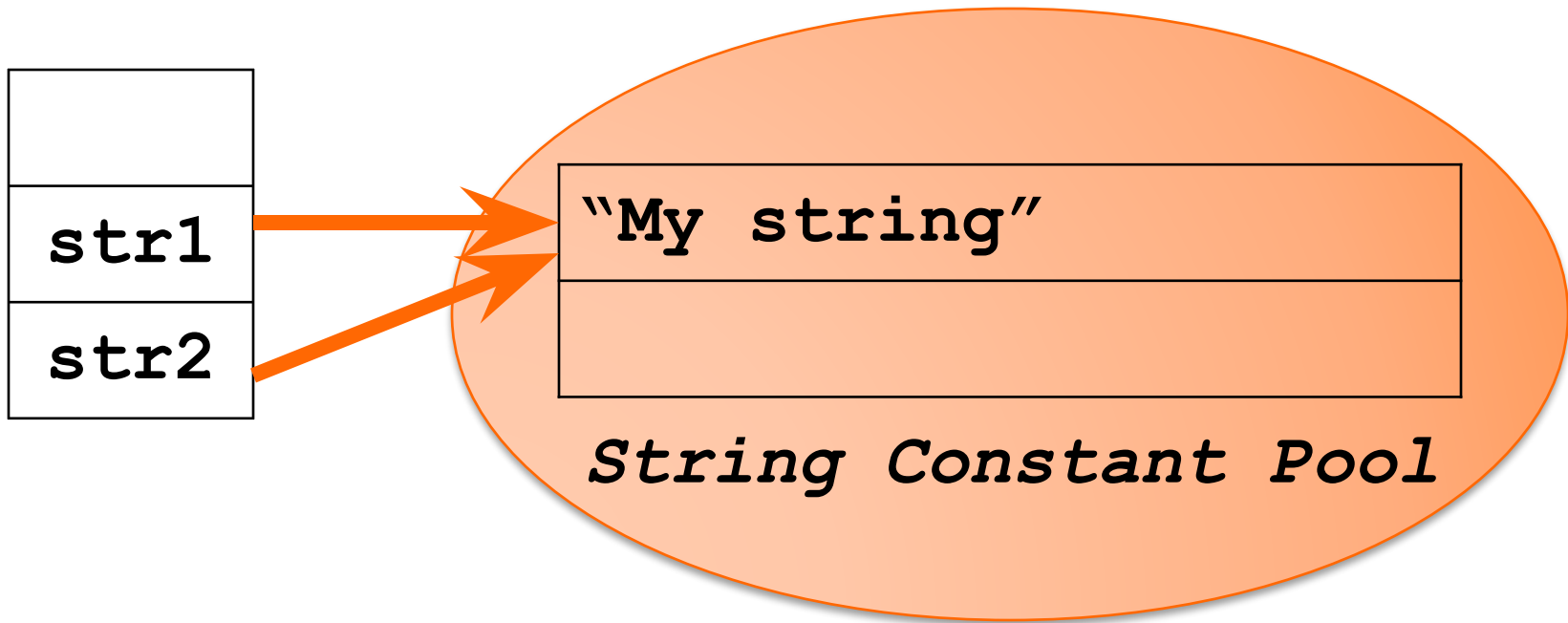
//Creating a string object

str1 = "My string";

//OR

//Declaring & creating String object

String str2 = "My string";



`str1 == str2` ☐ `true`

Creating a String:

Using *new* Operator

□ Sample String Object Instantiation:

//Declaration of String object variable

String str1; *;//the value of String
//object is null*

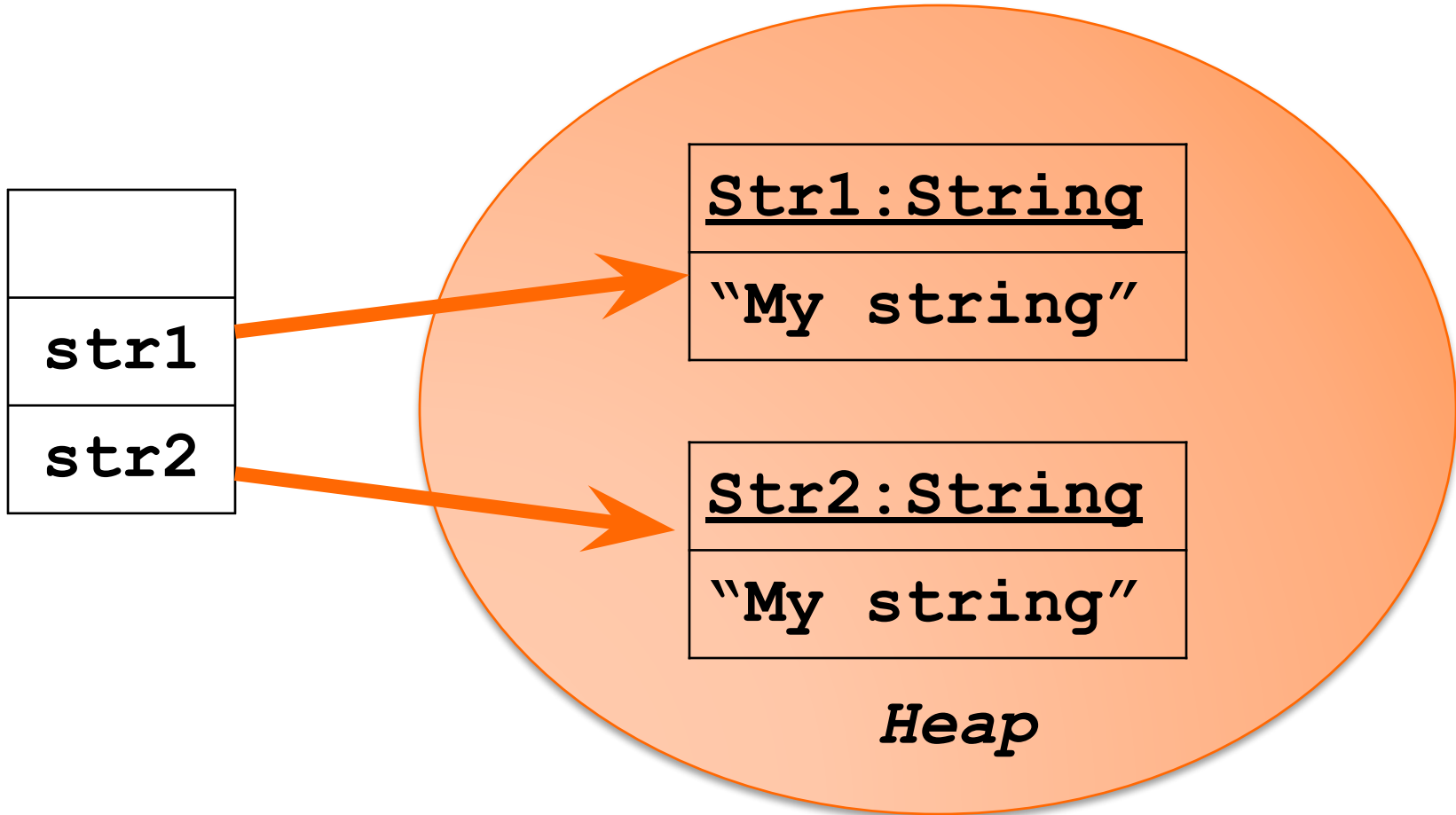
//Creating a string object

str1=new String("My string");

//OR

//Declaring & creating String object

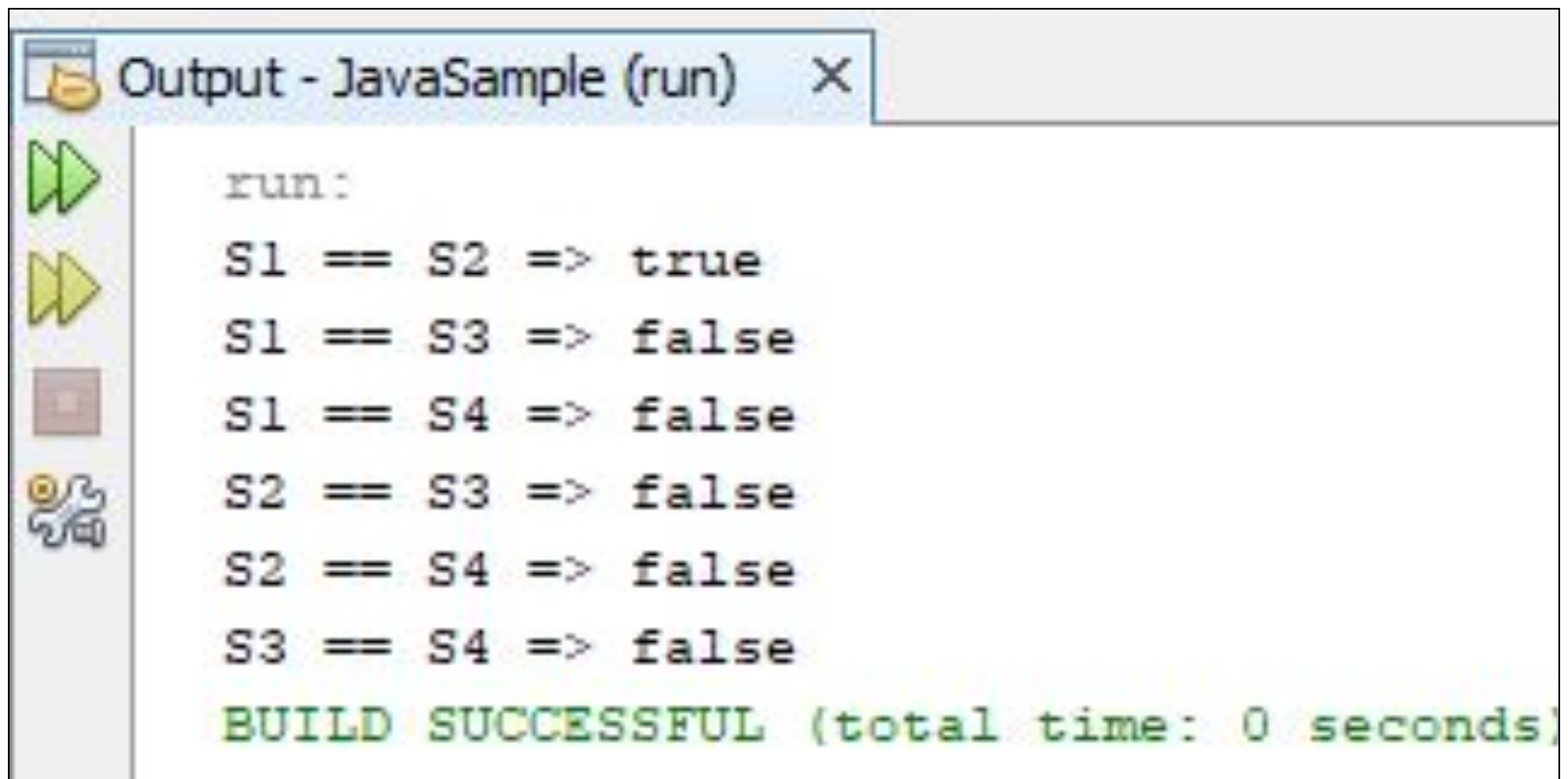
String str2=new String("My string");



`str1 == str2` ☐ `false`


```
String s1 = "Java Programming";  
String s2 = "Java Programming";  
String s3 = new String(s1);  
String s4 = new String(s2);
```

```
System.out.println("S1 == S2 => " + (s1==s2));  
System.out.println("S1 == S3 => " + (s1==s3));  
System.out.println("S1 == S4 => " + (s1==s4));  
System.out.println("S2 == S3 => " + (s2==s3));  
System.out.println("S2 == S4 => " + (s2==s4));  
System.out.println("S3 == S4 => " + (s3==s4));
```

A screenshot of an IDE's output window titled "Output - JavaSample (run)". The window has a light gray header bar with the title and a close button. On the left side, there is a vertical toolbar with four icons: a green double arrow, a yellow double arrow, a red square, and a circular icon with a network-like pattern. The main area of the window displays the output of a Java program in a monospaced font. The output consists of several lines of text, including logical expressions and their boolean results, followed by a green-colored status message.

```
run:
S1 == S2 => true
S1 == S3 => false
S1 == S4 => false
S2 == S3 => false
S2 == S4 => false
S3 == S4 => false
BUILD SUCCESSFUL (total time: 0 seconds)
```

THE PLUS (+) OPERATOR FOR STRING CONCATENATION

- Acts as string concatenation operator if one argument is String type
- Acts as arithmetic addition if both arguments are number type

String s;

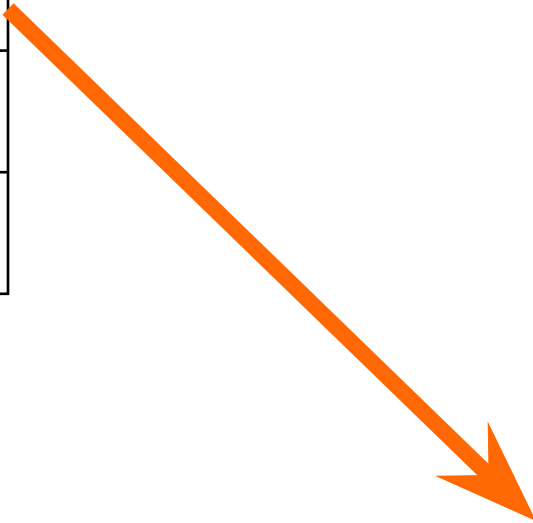
```
s = "junar" + '4' + 5 + 6.0; //s=junar456.0
```

```
s = '4' + 5 + 6.0 + "junar"; //s=63.0junar
```

```
s = '4' + "junar" + 5 +  
6.0); //s=4junar56.0
```

```
s = '4' + 5 + 6.0; //CE: double cannot  
//converted to String
```

s



"junar"
"junar4"
"junar45"
"junar456.0"
"63.0junar"
"4junar"
"4junar5"
"4junar56.0"

String Constant Pool

COMMON USED STRING INSTANCE METHODS

- ❑ `char charAt()`
- ❑ `int compareTo()`
- ❑ `String concat()`
- ❑ `boolean equals()` & `boolean equalsIgnoreCase()`
- ❑ `int indexOf()` & `int lastIndexOf()`
- ❑ `int length()`
- ❑ `String substring()`
- ❑ `boolean startsWith()` & `boolean endsWith()`
- ❑ `String toLowerCase()`
- ❑ `String toUpperCase()`
- ❑ `String trim()`

charAt() Method

- To extract a single character from a String
- *Form:* **char charAt(int index)**
- *Sample code:*

```
char ch = "Hello World".charAt(6) ; //ch= 'W'  
String str = "JAVA";  
for(int i=0; i<str.length(); i++)  
{  
    System.out.print(str.charAt(i) + ".");  
} //Output: J.A.V.A.
```

concat() Method

□ The *concat()* method is used to append a string at the end of the string

□ *Form: String concat(string)*

□ *Sample code:*

```
String name = "James";  
name = name.concat(" Gosling");  
System.out.println("I'm " + name + ".");
```

□ *Output:*

```
I'm James Gosling.
```

length() Method

- The length of a string is the number of characters that it contains
- To obtain this value, call the length() method
- Sample codes:

```
int len = "Hello".length(); //len=5
```

```
String str1 = " World";
```

```
len = str1.length(); //len=6
```

```
str = "Hello World"
```

```
System.out.println(str.length());
```

```
//the output is 11
```


indexOf() Method

- ▣ Searches for the first occurrence of a character or substring.
- ▣ The methods return the index at which the character or substring was found, or -1 on failure
- ▣ *Forms:*
 - `int indexOf(char ch)`
 - `int indexOf(String s)`
 - `int indexOf(char ch, int startIndex)`
 - `int indexOf(String s , int startIndex)`

▣ *Sample code:*

```
String word = "Mississippi";  
int index = word.indexOf(i); //index=1  
index = word.indexOf("si"); //index=3  
index = word.indexOf('i',5); //index=7  
index = word.indexOf("si",5); //index=6
```

lastIndexOf() Method

- ▣ Searches for the last occurrence of a character or substring.
- ▣ The methods return the index at which the character or substring was found, or -1 on failure
- ▣ *Forms:*
 - `int lastIndexOf(char ch)`
 - `int lastIndexOf(String s)`
 - `int lastIndexOf(char ch, int startIndex)`
 - `int lastIndexOf(String s , int startIndex)`

▣ *Sample code:*

```
String word = "Mississippi";  
int index = word.lastIndexOf(i);  
//index=10  
index = word.lastIndexOf("si");  
//index=6  
index = word.lastIndexOf('i',5);  
//index=4  
index = word.lastIndexOf("si",5);  
//index=3
```

substring() Method

- Extracts substring from a string

- *Forms:*

- String substring(int startIndex)

- *Return the sub string of the string from startIndex*

- String substring(int startIndex, int lastIndex)

- *Return the sub string of the string from startIndex to lastIndex-1*

```
public static void main(String[] args)
{
    String word = "Java Programming";
    //Get the java word
    int index = word.indexOf(" ");
    String javaString= word.substring(0, index);
    //remove the "Java"
    word = word.substring(index+1); //word="Programming"
    System.out.println(word + " using " + javaString);
}
```

Output - StringDemo (run)



run:



Programming using Java

replace() Method

- ▣ Replaces all occurrences of one character or character sequence in the invoking string with another character or character sequence, respectively
- ▣ *Forms:*
 - String replace (char origChar, char newChar)
 - String replace(String origStr, String newStr)

```
public static void main(String[] args)
{
    String sentence = "Java programming, this is not easy or hard.";
    sentence = sentence.replace('a', '@');
    System.out.println(sentence);
    System.out.println(sentence.replace("is", "was"));
}
```

Output - StringDemo (run)

run:

J@v@ progr@mming, this is not e@sy or h@rd.

J@v@ progr@mming, thwas was not e@sy or h@rd.

trim() Method

- ▣ Returns a copy of the invoking string from which any leading and trailing whitespace has been removed
- ▣ *Forms:*
 - String trim()

```
public static void main(String[] args)
{
    String phrase = "    Java    programming    ";
    System.out.println("Original: \"" + phrase + "\"");

    phrase = phrase.trim();
    System.out.println("Trimmed: \"" + phrase + "\"");
}
```

Output - StringDemo (run)

run:

Original: " Java programming "

Trimmed: "Java programming"

toLowerCase() Method & toUpperCase Method

- ▣ *toLowerCase()* method converts all the characters in a string from uppercase to lowercase
 - *Forms:* String toLowerCase()
- ▣ *toUpperCase()* method converts all the characters in a string from lowercase to uppercase
 - *Forms:* String toUpperCase()

Note: Non-alphabetical characters, such as digits, are unaffected

```
public static void main(String[] args)
{
    String phrase = "Java Programming";
    System.out.println("Original: " + phrase );

    String lower = phrase.toLowerCase();
    System.out.println("Lower Case:" + lower );

    String upper = phrase.toUpperCase();
    System.out.println("Upper Case:" + upper );
}
```

Output - StringDemo (run)

run:

Original: Java Programming

Lower Case:java programming

Upper Case:JAVA PROGRAMMING

startsWith() Method & endsWith() Method

▣ *startsWith()* method determines whether a given string begins with a specified string

● *Forms:*

▣ boolean startsWith(String str)

▣ boolean startsWith(String str,
int startIndex)

▣ *endsWith()* method determines whether the a given string ends with a specified string

● *Forms:* boolean endsWith(String str)

```
public static void main(String[] args)
{
    String s = "JavaProgramming";
    if(s.startsWith("Java"))
    {
        System.out.println("The string starts with \"Java\"");
        if (s.startsWith(" ",4))
        {
            System.out.println("With space after \"Java\"");
        }
    }

    if(s.endsWith("ing"))
    {
        System.out.println("The string ends with \"ing\"");
    }
}
```

Output - StringDemo (run)



run:



The string starts with "Java"

The string ends with "ing"

equals() Method & equalsIgnoreCase() Method

- ▣ ***equals()*** method used to compare two strings for equality
 - *Forms:* boolean equals(Object str)

- ▣ ***equalsIgnoreCase()*** method used to perform a comparison that ignores case differences
 - *Forms:* boolean equalsIgnoreCase(Object str)


```
public static void main(String[] args)
{
    String s = "Java";
    if(s.equals("JAVA"))
    {
        System.out.println(s + " is equal to JAVA");
    }
    else
    {
        System.out.println(s + " is not equal to JAVA");
    }
    if(s.equalsIgnoreCase("JAVA"))
    {
        System.out.println("Ignore Case:" + s + " is equal to JAVA");
    }
}
```

Output - StringDemo (run)



run:



Java is not equal to JAVA
Ignore Case:Java is equal to JAVA

compareTo() Method

- ❑ Used to determine if the given string is *less than*, *equal to*, or *greater than* to the other string
- ❑ A string is *less than* another if it comes before the other in chronological order (***return negative value***)
- ❑ A string is *greater than* another if it comes after the other in chronological order (***return positive value***)
- ❑ A string is equal to another ***returns 0***
- ❑ *Forms:* `int compareTo(Object str)`

```
public static void main(String[] args)
{
    String s1 = "banana";
    String s2 = "apple";

    if(s1.compareTo(s2)>0)
    {
        System.out.println(s2 + " " + s1);
    }
    else if(s1.compareTo(s2)<0)
    {
        System.out.println(s1 + " " + s2);
    }
}
```

Output - StringDemo (run)



run:

apple banana

The String.join() Method

- Concatenate two or more strings, separating each string with a delimiter, such as a space or a comma
- *Forms:*
 - static String join(String delim, String s1, String s2, ..., String sN)

```
public static void main(String[] args)
{
    String result = String.join(" ", "JAVA", "PROGRAMMING");
    System.out.println(result);

    result = String.join("-", "JAVA", "PROGRAMMING", "IS", "FUN");
    System.out.println(result);
}
```

Output - StringDemo (run)



JAVA PROGRAMMING



JAVA-PROGRAMMING-IS-FUN

The String.format() Method

- ▣ Used to create a string with formatted numeric values
- ▣ *Forms:*
 - static String format(String str, val1, val2, ..., valN)

```
public static void main(String[] args)
{
    for(int i = 1; i<=10; i++)
    {
        String result = String.format("Number[%02d] = %.3f",i, 1.0/i);
        System.out.println(result);
    }
}
```

Output - StringDemo (run)



run:



Number[01] = 1.000



Number[02] = 0.500



Number[03] = 0.333

Number[04] = 0.250

Number[05] = 0.200

Number[06] = 0.167

Number[07] = 0.143

Number[08] = 0.125

Number[09] = 0.111

Number[10] = 0.100

Accepting String User Input: next() Method of Scanner Class

```
public static void main(String[] args)
{
    Scanner input = new Scanner (System.in);
    System.out.print("Enter a word: ");
    String word = input.next();
    System.out.println("Word: " + word);
}
```

Output - StringDemo (run)	
	run:
	Enter a word: Java Programming Language
	Word: Java

Accepting String User Input: nextLine() Method of Scanner Class

```
public static void main(String[] args)
{
    Scanner input = new Scanner (System.in);
    System.out.print("Enter a word: ");
    String line = input.nextLine();
    System.out.println("Sentence: " + line);
}
```

Output - StringDemo (run)



run:



Enter a word: Java Programming Language
Sentence: Java Programming Language

THE STRINGBUILDER CLASS

- ❑ StringBuilder object is a mutable sequence of characters
- ❑ StringBuilder is exactly the same with StringBuffer (including constructors and methods), but with no guarantee of ***synchronization (not thread-safe)***
- ❑ This class is designed for use as a drop-in replacement for StringBuffer in places where the string buffer was being used by a single thread (as is generally the case)
- ❑ It is recommended that this class be used in preference to StringBuffer as it will be faster under most implementations

- ## □ API Documentation:

<https://docs.oracle.com/javase/8/docs/api/java/lang/StringBuilder.html>

- Common constructors use to create a `StringBuffer` object:

- `StringBuilder sb = new StringBuilder();`
- `StringBuilder sb = new StringBuilder
(int init_cap);`
- `StringBuilder sb = new StringBuilder
(String s);`

COMMON USED StringBuilder METHODS

- ❑ `int length();`
- ❑ `int capacity();`
- ❑ `char charAt();`
- ❑ `void setCharAt();`
- ❑ `StringBuilder append();`
- ❑ `StringBuilder insert();`
- ❑ `StringBuilder replace()`
- ❑ `String substring()`
- ❑ `String toString()`

STRING VS STRINGBUILDER

- ❑ String is recommended to use if the content is fixed and won't change frequently
- ❑ StringBuilder is recommended to use if the content will change frequently and thread-safety is not required

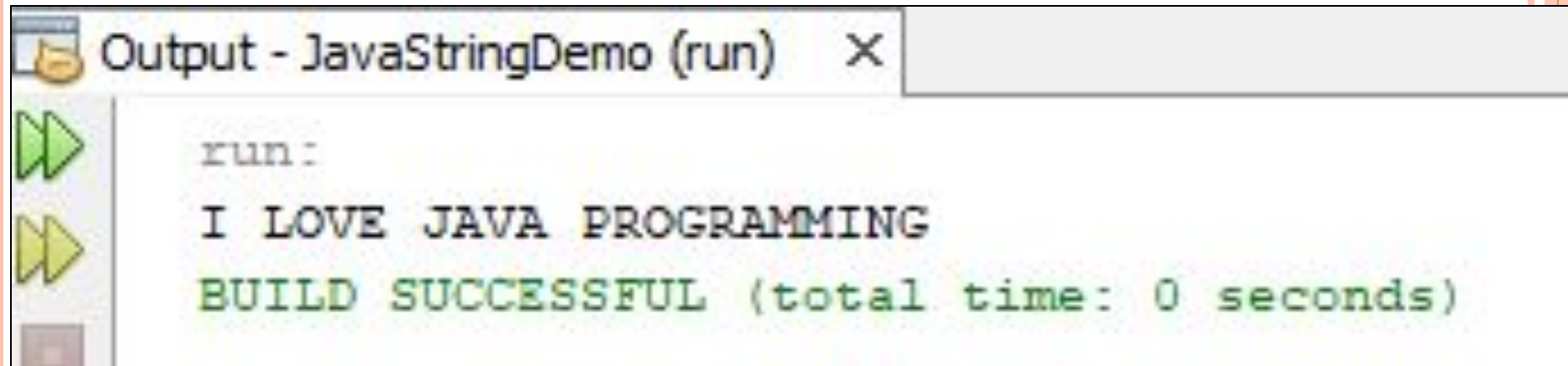
METHOD CHAINING

- ❑ For most of the methods in `String`, `StringBuffer`, and `StringBuilder` the return type is same type
- ❑ Method chaining is calling another method after calling a method

`obj.method1().method2().method3() . . . .`

- ❑ In method chaining, all methods will be evaluated from left to right

```
public static void main(String[] args)
{
    String s = "I hate Java";
    s = s.toUpperCase().replace("HATE", "LOVE").concat(" PROGRAMMING");
    System.out.println(s);
}
```



Output - JavaStringDemo (run) X

```
run:
I LOVE JAVA PROGRAMMING
BUILD SUCCESSFUL (total time: 0 seconds)
```

WRAPPER CLASSES

- Main objectives of Wrapper Classes:
 - To wrap primitives into object form (to handle primitives just like objects)
 - To define several utility functions which are required for the primitives

- Wrapper Classes:

Byte	Long	Character
Short	Float	Boolean
Integer	Double	

- Number Type: Byte, Short, Integer, Long, Float, & Double

Byte Class

- The Byte class wraps a value of *primitive type byte* in an object
- This class provides several methods for converting a byte type to a String and a String to a byte type, as well as other constants and methods useful when dealing with a byte type
- API documentation:
(<https://docs.oracle.com/javase/8/docs/api/java/lang/Byte.html>)

- ❑ For the constructor you can pass a byte value or a String (accepted integral literal format)
- ❑ If the String value is invalid a runtime error will occur (NumberFormatException)

```
Byte b1 = new Byte("10");  
Byte b2 = new Byte((byte) 10);  
Byte b3 = new Byte("1A"); //RE: NumberFormatException  
Byte b4 = new Byte("300"); //RE: NumberFormatException  
                                //Out-of-Range  
  
//Accessing the static field of Byte Class  
byte b5 = Byte.MAX_VALUE;  
//Calling a static method of Byte class  
byte b6 = Byte.valueOf("15");  
//Calling the instance method of Byte object  
int i1 = b1.compareTo(b6); //i = -5
```

Short Class

- ❑ The Short class wraps a value of *primitive type short* in an object
- ❑ This class provides several methods for converting a short type to a String and a String to a short type, as well as other constants and methods useful when dealing with a short type
- ❑ API documentation:
(<https://docs.oracle.com/javase/8/docs/api/java/lang/Short.html>)

- ❑ For the constructor you can pass a short value or a String (accepted integral literal format)
- ❑ If the String value is invalid a runtime error will occur (NumberFormatException)

```
Short s1 = new Short("1300");
Short s2 = new Short((short)100);
Short s3 = new Short("1A"); //RE: NumberFormatException
Short s4 = new Short("99999"); //RE: NumberFormatException
                                     //Out-of-Range

//Accessing the static field of Short Class
short s5 = Short.MIN_VALUE;
//Calling a static method of Short class
short s6 = Short.reverseBytes(s5);
//Calling the instance method of Short object
int i = s1.intValue(); //i = 1300
```

Integer Class

- ❑ The Integer class wraps a value of *primitive type int* in an object
- ❑ This class provides several methods for converting an int type to a String and a String to a int type, as well as other constants and methods useful when dealing with an int type
- ❑ API documentation:
(<https://docs.oracle.com/javase/8/docs/api/java/lang/Integer.html>)

- ❑ For the constructor you can pass an int value or a String (accepted integral literal format)
- ❑ If the String value is invalid a runtime error will occur (NumberFormatException)

```
Integer i1 = new Integer("1300");  
Integer i2 = new Integer(0x1F);  
Integer i3 = new Integer("1A");//RE: NumberFormatException  
  
//Accessing the static field of Integer Class  
int i5 = Integer.BYTES;//i5 = 4  
//Calling a static method of Integer class  
int i6 = Integer.bitCount(i2);//i6 = 5  
//Calling the instance method of Integer object  
String s = i2.toString();//s = "31"
```

Long Class

- The Long class wraps a value of *primitive type long* in an object
- This class provides several methods for converting a long type to a String and a String to a long type, as well as other constants and methods useful when dealing with a long type
- API documentation:
(<https://docs.oracle.com/javase/8/docs/api/java/lang/Long.html>)

- ❑ For the constructor you can pass an long value or a String (accepted integral literal format)
- ❑ If the String value is invalid a runtime error will occur (NumberFormatException)

```
//Creating Long object
Long l = new Long("9089898323289656");

//accessing Long Class static variable
long max = Long.MAX_VALUE;
//max = 9223372036854775807

//calling Long Class static method
String hexa = Long.toHexString(90989032);
System.out.println(hexa.toUpperCase()); //56C61E8
//calling Long object instance method
int i = l.intValue(); //i = -526735816
```

Float Class

- The Float class wraps a value of *primitive type float* in an object
- The class provides several methods for converting a float type to a String and a String to a float type, as well as other constants and methods useful when dealing with a float type
- API documentation:
(<https://docs.oracle.com/javase/8/docs/api/java/lang/Float.html>)

- ❑ For the constructor you can pass any numeric type value or a String (accepted number type format)
- ❑ If the String value is invalid a runtime error will occur (NumberFormatException)

```
Float f1 = new Float("13.0");  
Float f2 = new Float(10/3.0);  
Float f3 = new Float(17.5f);  
Float f4 = new Float("1,000.0");//RE: NumberFormatException  
  
//Accessing the static field of Float Class  
int i5 = Float.MAX_EXPONENT;//i5 = 127  
//Calling a static method of Float class  
int i6 = Float.floatToIntBits(1.5f);//i6 = 1069547520  
//Calling the instance method of Float object  
boolean b = f2.isInfinite();//b = false
```

Double Class

- ❑ The Double class wraps a value of the *primitive type double* in an object
- ❑ The class provides several methods for converting a double type to a String and a String to a double type, as well as other constants and methods useful when dealing with a double type.
- ❑ API documentation:
(<https://docs.oracle.com/javase/8/docs/api/java/lang/Double.html>)

- ❑ For the constructor you can pass any numeric type value or a String (accepted number type format)
- ❑ If the String value is invalid a runtime error will occur (NumberFormatException)

```
Double d1 = new Double("10.5");  
Double d2 = new Double("10.5f");  
Double d3 = new Double("10.5d");  
Double d4 = new Double("10.5s");//RE: NumberFormatException  
Double d5 = new Double(109);//Accept any numeric value  
  
//Accessing the static field of Float Class  
int size = Double.SIZE;//size = 64  
//Calling a static method of Float class  
double max_val = Double.max(d5, d1);//max_val = 109.0  
//Calling the instance method of Float object  
String s = d3.toString();//s = "109.0"
```

Character Class

- ❑ The Character class wraps a value of the *primitive type char* in an object.
- ❑ An object of class Character contains a single field whose type is char.
- ❑ The class provides a large number of static methods for determining a character's category (lowercase letter, digit, etc.) and for converting characters from uppercase to lowercase and vice versa
- ❑ API documentation:
(<https://docs.oracle.com/javase/8/docs/api/java/lang/Character.html>)

- ❑ For the constructor you can pass a character value only

```
//only accept a char as argument for constructor
Character c1 = new Character('A');

//Accessing the static field of Float Class
int i5 = Character.LINE_SEPARATOR; //i5 = 13
//Calling a static method of Float class
boolean b = Character.isUpperCase('B'); //b = true
//Calling the instance method of Float object
String s = c1.toString(); //s = "A"
```

Boolean Class

- The Boolean class wraps a value of the *primitive type boolean* in an object
- The class provides many methods for converting a boolean type to a String and a String to a boolean type, as well as other constants and methods useful when dealing with a boolean type
- API documentation:
(<https://docs.oracle.com/javase/8/docs/api/java/lang/Boolean.html>)

- ❑ For the constructor you can pass a boolean type or a String of “true” or “false” (Not case-sensitive)
- ❑ If the String is invalid the boolean value is false

```
Boolean b1 = new Boolean(true);  
Boolean b2 = new Boolean(false);  
Boolean b3 = new Boolean("TRUE");//true  
Boolean b4 = new Boolean("100");//false  
Boolean b5 = new Boolean("False");//false  
  
//Accessing the static field of Boolean Class  
boolean b6 = Boolean.TRUE;//b6 = true  
//Calling a static method of Boolean class  
boolean b7 = Boolean.logicalAnd(b3, b5);//false  
//Calling the instance method of Boolean object  
boolean b8 = b4.equals(b7);//true
```

UTILITY METHODS

- ❑ `valueOf()` Method
- ❑ `xxxValue()` Method (`xxx` is any primitive type)
- ❑ `parseXxx()` Method (`Xxx` is any primitive type)
- ❑ `toString()` Method

static <class_obj> valueOf() Method

- A static method use to create a wrapper object for the given primitive type or String as alternative to construct
- Returns the indicated wrapper class object
- Forms:
 - **valueOf (primitive_type)**
 - *primitive_type: byte, short, int, float, double, or boolean*
 - **valueOf (String)**
 - *String is acceptable format for numeric type or else Runtime Error: NumberFormatException*
 - **valueOf (String, int)**
 - *for integral type only*

```
Boolean bool = Boolean.valueOf("True");
```

```
Double d = Double.valueOf("102.0F");
```

```
Float f = Float.valueOf("25.76D");
```

```
Integer i = Integer.valueOf("1000");
```

```
//Using valueOf(String, int)
```

```
//int is a radix value of number systems
```

```
Integer i2 = Integer.valueOf("FFFF",16);//hexa
```

```
Short s = Short.valueOf("7362",8);//octal
```

```
Byte b = Byte.valueOf("11110",2);//binary
```

<type> xxxValue() Method

- An instance method use to convert the wrapper object to primitive
- Every number type wrapper class contains the following xxxValue methods:
 - byte byteValue()
 - short shortValue()
 - int intValue()
 - long longValue()
 - float floatValue()
 - double doubleValue()

```
Integer I = new Integer("150");  
byte b = I.byteValue(); // b = -106  
short s = I.shortValue(); // s = 150  
int i = I.intValue(); // i = 150  
long l = I.longValue(); // l = 150  
float f = I.floatValue(); // f = 150.0  
double d = I.doubleValue(); // d = 150.0
```

static <p_type> parseXxx() Method

- A static method that convert the String to corresponding primitive
- Every Wrapper class had this method except for Character class
- Forms:
 - parseXxx(String)
 - *String is acceptable format for numeric type or else Runtime Error: NumberFormatException*
 - parseXxx(String, int)
 - *For integral type only*

```
boolean b = Boolean.parseBoolean("False");//b = false
double d = Double.parseDouble("10.50");// d = 10.5
float f = Float.parseFloat("10.50");//f = 10.5
int i1 = Integer.parseInt("100");//i1 = 100
short s1 = Short.parseShort("300");//s1 = 300
byte b1 = Byte.parseByte("10");//b1 = 10

//using parserXxx(String, int)
int i2 = Integer.parseInt("100", 2);//i2 = 4
short s2 = Short.parseShort("FAB", 16);//s = 4011
byte b2 = Byte.parseByte("65", 8);//b2 = 53
```

String toString() Method

- An instance method that convert wrapper objet to String
- Every Wrapper class had this method except for Character class
- Forms:
 - toString()


```
Integer i = Integer.valueOf("100", 2);  
String i_str = i.toString(); //s = "4"
```

```
Double d = new Double("18.0e5");  
String d_str = d.toString(); //d_str = "1800000.0"
```

```
Boolean b = Boolean.valueOf("TrUe");  
String b_str = b.toString(); //b_str = "true"
```


MORE ABOUT WRAPPER CLASSES

- ❑ All Wrapper Classes are Final **Class** (*cannot be used as parent class*)
- ❑ All Wrapper objects are ***immutable object*** (can't modified the existing object, trying to perform any changes will create new object)
- ❑ Two automatic conversion done by compiler for primitive type & wrapper object:
 1. Autoboxing
 2. Autounboxing

Autoboxing Concept

- Automatic conversion of primitive type to wrapper object by a compiler

Example:

```
Integer i = 10;
```

After compilation:

```
Integer i = Integer.valueOf(10);
```

- Autoboxing concept is implemented by using *valueOf()* method

Autounboxing Concept

- Automatic conversion of wrapper object primitive type by a compiler

Example:

```
Integer I = new Integer(10);
```

```
int i = I;
```

After compilation:

```
int i = I.intValue();
```

- Autounboxing concept is implemented by using *xxxValue()* method

CHAPTER 1

JAVA

PROGRAMMING

FUNDAMENTALS

END

256

